

COMPUTAÇÃO GRÁFICA



Murillo Rodrigo Petrucelli Homem
Departamento de Computação

O que buscamos...

- compreender a organização e as funcionalidades de **sistemas gráficos**;
- compreender as estruturas de baixo nível necessárias para a construção e manipulação de sistemas gráficos;
- identificar os principais problemas associados à **síntese de imagens** considerando os diferentes sistemas de referências existentes;
- compreender e identificar todas as etapas do "**pipeline**" de **visualização**;
- justificar o uso de determinados algoritmos em cada etapa do "*pipeline*" de visualização;
- implementar **abordagens básicas** na solução de problemas em computação gráfica.

Sistemas gráficos

- Com a finalidade de padronizar a construção de aplicativos que utilizam recursos gráficos, surge a ideia do desenvolvimento dos chamados **Sistemas Gráficos** (SG);
- Exemplo: X-Window (desenvolvido no MIT, em 1984) – Protocolo para GUI's (*Graphical User Interfaces*) para ambientes baseados no UNIX (Linux, Minix etc);
- Mais recentemente, o *Wayland*, um protocolo de servidor de exibição para sistemas de janela, projetado para ser o sucessor do *X-Window System* – um protocolo de comunicação entre um *compositor* (que desenha na tela do computador) e os seus clientes (aplicativos), facilitando a troca de informações e a renderização de conteúdo.

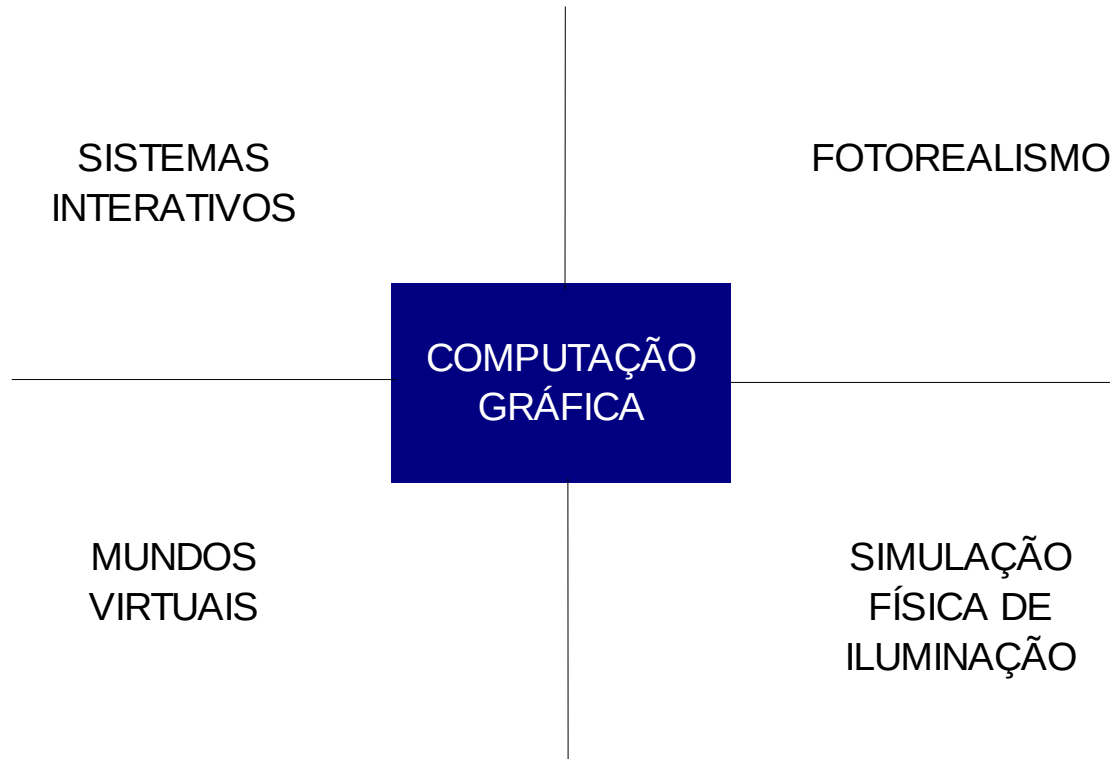
Características desejáveis

- **Minimalidade:** um SG deve implementar uma API (*Application Programmer Interface*) mínima, mas funcional;
- **Portabilidade:** um SG deve ser independente do dispositivo;
- **Separabilidade:** a arquitetura do SG deve isolar os detalhes de implementação da API gráfica;
- **Disponibilidade:** deve suportar a maioria das plataformas existentes.

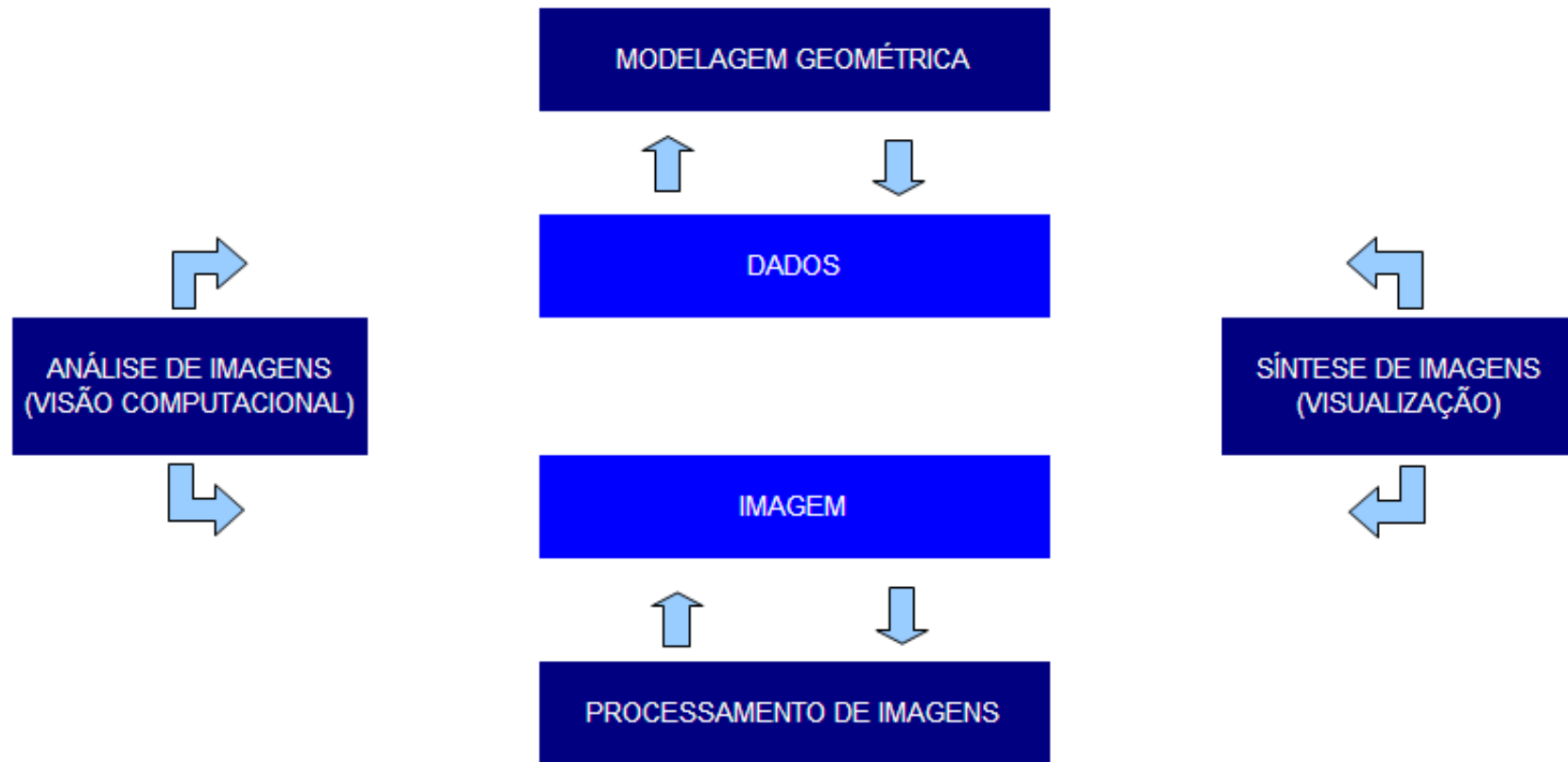
Definição de Computação Gráfica

- A Computação Gráfica é a área da Ciência da Computação que estuda os processos computacionais para a **geração, manipulação e interpretação** de modelos geométricos (de *objetos digitais*);
- Curiosidade: o termo “computação gráfica”, foi utilizado pela primeira vez por *Verne Hudson*, durante a coordenação de um projeto para a Boeing, em 1959.

Escopo da Computação Gráfica



Escopo da Computação Gráfica



Entrada e Saída em CG

- Dependendo da natureza dos seus dados (analógica ou digital), os dispositivos gráficos (hardware) de entrada e saída utilizados em computação gráfica podem ser classificados em duas grandes linhas:
 - Dispositivos vetoriais;
 - Dispositivos matriciais.
- Usualmente uma boa combinação consiste em utilizar dispositivos vetoriais de entrada gráfica, como o mouse ou o tablet, por exemplo, e dispositivos matriciais de saída, como monitores CRT ou LCD e impressoras a laser ou jato-de-tinta.

Entrada e Saída em CG

Dispositivos vetoriais

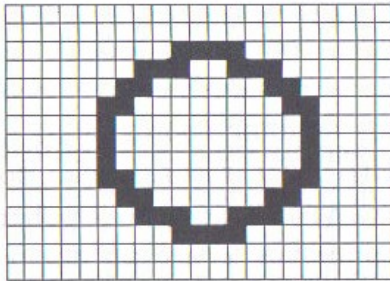
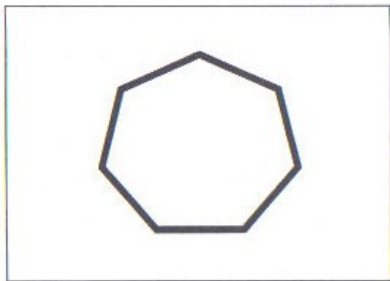
- Desenharam figuras traçando sequências de segmentos de retas (vetores);
- Estes dispositivos permitem o traçado perfeito de um segmento de reta entre dois pontos, sem distorções, dentro de uma malha finita de pontos definida por suas superfícies de exibição.

Dispositivos matriciais

- Desenharam figuras pelo preenchimento de entradas retangulares em uma matriz (*pixels*);
- Preenchem entradas em uma grade finita entradas em uma grade finita – assim segmentos de reta são aproximados por uma sequência de entradas próximas.

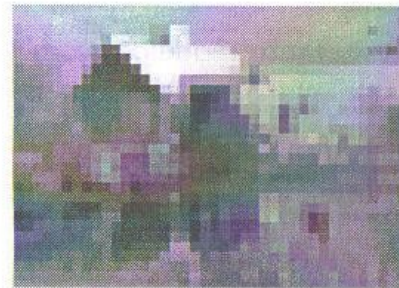
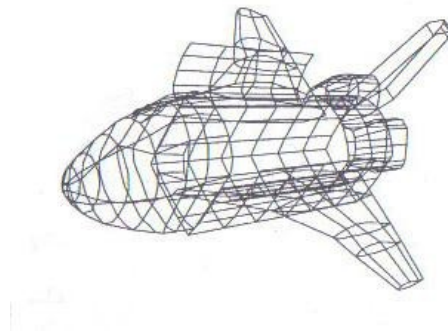
Representações

(vetoriais x matriciais)

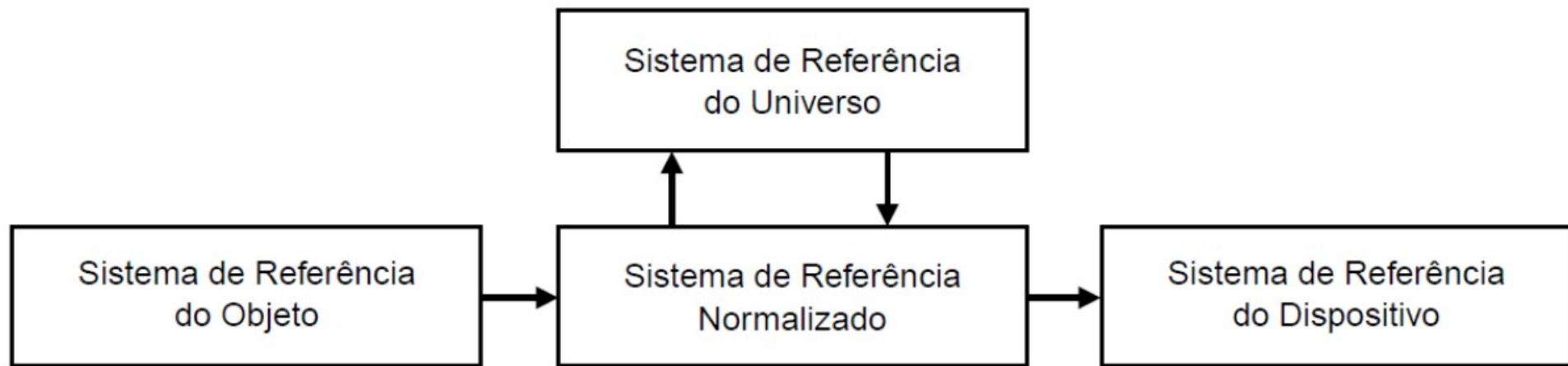


- Pontos e segmentos de reta são perfeitamente ajustados em dispositivos vetoriais (precisão nos dados);

- Devido a natureza discreta da maioria dos dispositivos de saída da atualidade, a representação em uma grade matricial se torna mais apropriada.



Sistemas de Referências



Sistemas de Referências

- Um determinado sistema de coordenadas é denominado de Sistema de Referência se for um sistema de coordenadas (cartesianas) para “alguma finalidade específica”;
- Ao definirmos um sistema de coordenadas de referência, devemos especificar dois aspectos principais: a unidade de referência básica e os limites extremos dos valores aceitos para descrever os objetos.

Sistemas de Referências

Sistema de Referência do Universo (SRU)



É chamado de coordenadas do **universo**, ou do **mundo**, o sistema de referência utilizado para descrever os objetos em termos das coordenadas (**pontos e segmentos de reta**) utilizadas pelo usuário em determinada aplicação.

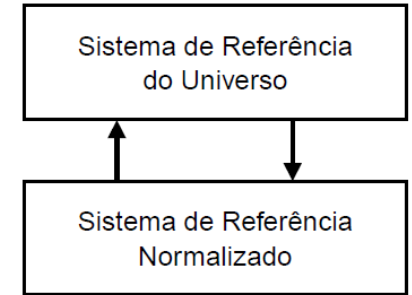
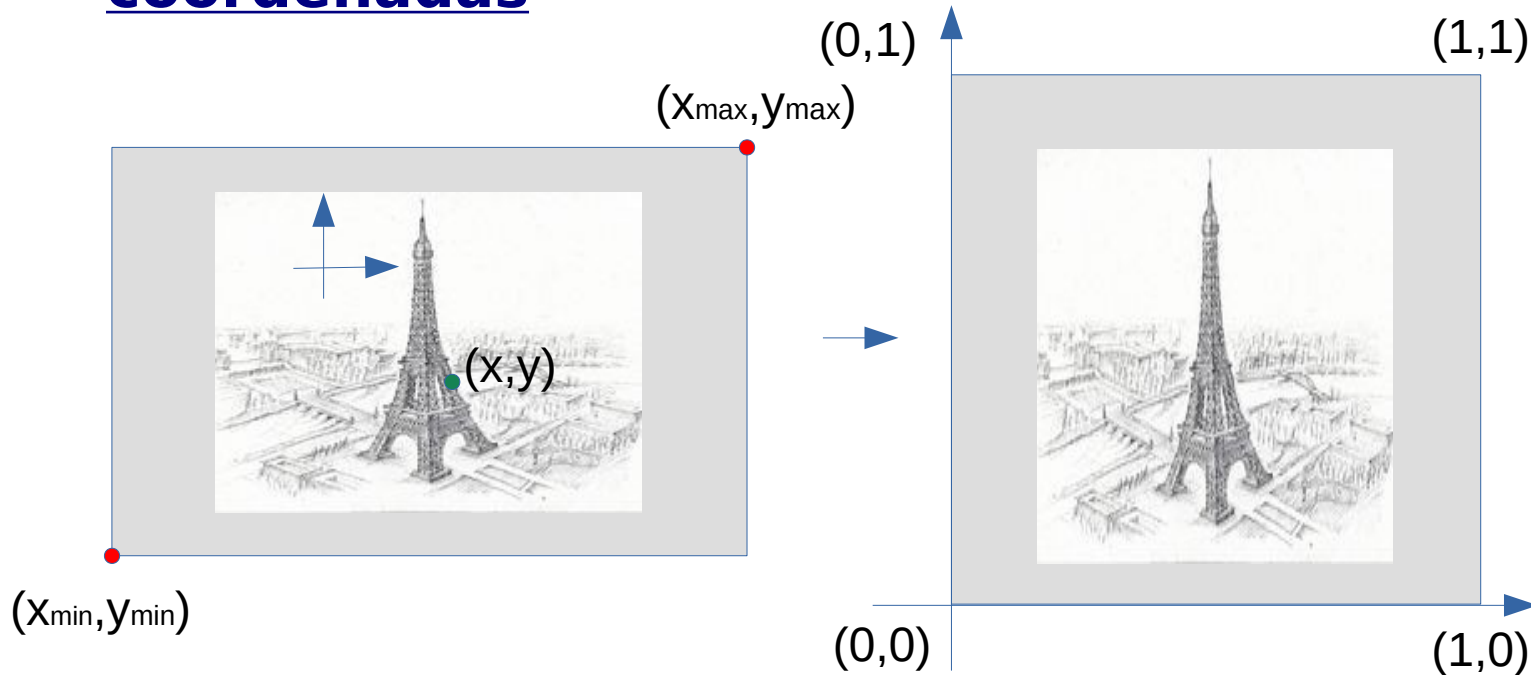
Sistemas de Referências

Sistema de Referência Normalizado (SRN)

- Esse sistema trabalha com as coordenadas normalizadas, isso é, com valores entre 0 e 1;
- O SRN serve como um sistema de referência intermediário entre o SRU e o SRD;
- Sua principal aplicação é tornar a geração das imagens independente do dispositivo, pois as coordenadas do universo são convertidas para um sistema de coordenadas padrão normalizado.

Sistemas de Referências

Mapeamento das coordenadas



Sistemas de Referências

- Os parâmetros que definem o intervalo de valores de x e y , $x_{\min}$, $y_{\min}$, $x_{\max}$ e $y_{\max}$, definem uma área retangular no espaço bidimensional, denominada de janela de visualização;
- A transformação de coordenadas do usuário (x, y) para *coordenadas normalizadas*, denominada transformação de visualização, é dada por:

$$ncx = (x - x_{\min}) / (x_{\max} - x_{\min})$$

$$ncy = (y - y_{\min}) / (y_{\max} - y_{\min})$$

Sistemas de Referências

- **Importante:** não confundir a **janela de visualização** (ou simplesmente, janela), com o conceito de janela em um Sistema Operacional (SO);
- A *janela de visualização* em Computação Gráfica é definida no Sistema de Referência do Universo (SRU);
- Dessa forma, as coordenadas são pares ordenados de números reais, descrevendo um ponto no mundo, sempre em relação a um sistema de coordenadas cartesianas;
- O termo janela comumente usado em SO será referido como *viewport*, definida no SRD.

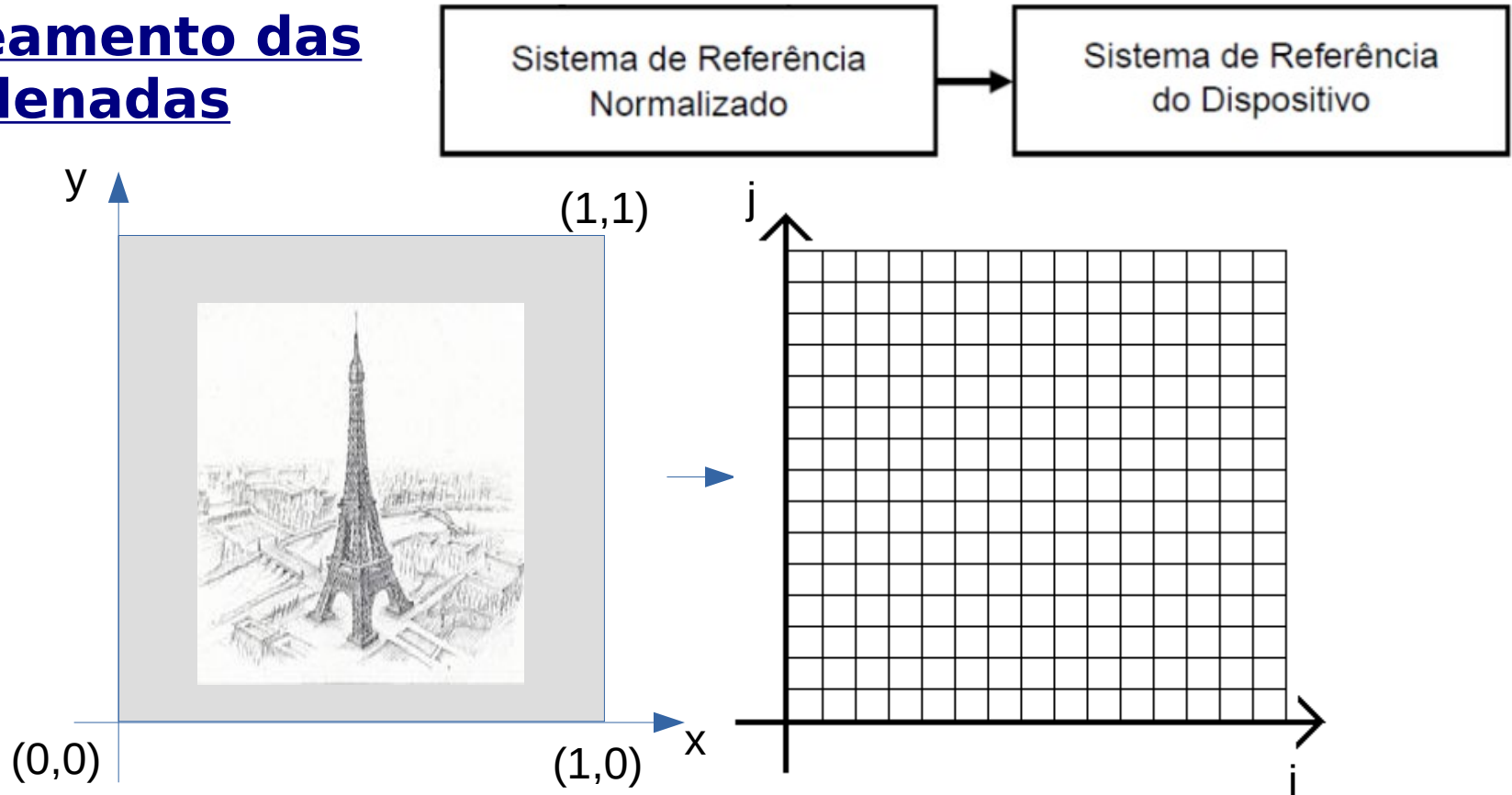
Sistemas de Referências

Sistema de Referência do Dispositivo (SRD)

- Utiliza coordenadas que podem ser fornecidas diretamente para um dado dispositivo de saída específico.
- Por exemplo, em um vídeo esses valores podem ser o número máximo de *entradas* que podem ser acesas (640×480 etc.) ou podem indicar a resolução especificada em determinada configuração do sistema operacional.

Sistemas de Referências

Mapeamento das coordenadas



Sistemas de Referências

- O mapeamento de coordenadas normalizadas (reais) para coordenadas do dispositivo (inteiros não negativos) é linear e pode ser considerada como:

$$dcx = \text{round}(ncx * (ndh - 1))$$

$$dcy = \text{round}(ncy * (ndv - 1))$$

- Essas equações fazem o mapeamento de **toda a janela** (no SRU) *para todo* o SRD;
- Para generalizá-las, é necessário ajustá-las para incorporar um tamanho menor no mapeamento para o SRD e um deslocamento a partir da origem.

Sistemas de Referências

- Com a generalização, podemos criar *viewports* de qualquer tamanho (respeitadas as dimensões do SRD) e em qualquer lugar, mesmo que sobrepostas ou parcialmente visíveis);

- Podemos escrever,

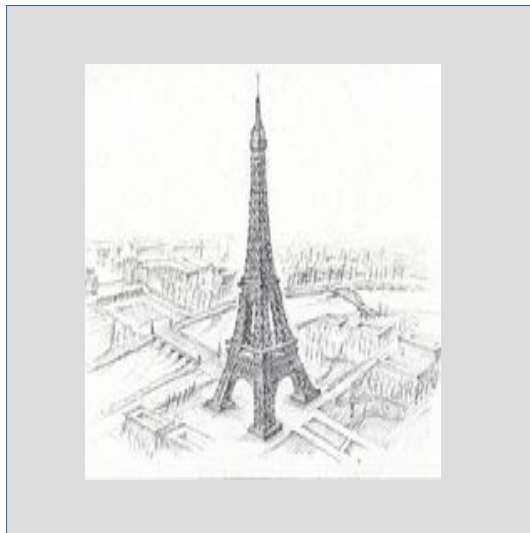
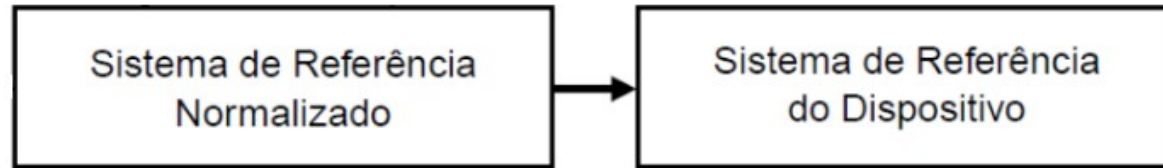
$$dcx = off_set_x + \text{round}(tam_x * (ndh - 1))$$

$$dcy = off_set_y + \text{round}(tam_y * (ndv - 1))$$

- Sendo *off_set_x* e *off_set_y* os deslocamentos da *viewport* nas direções *x* e *y*, respectivamente, em relação a origem do sistema de coordenadas do SRD e *tam_x* e *tam_y* o tamanho da *viewport* nas direções *x* e *y*, respectivamente.

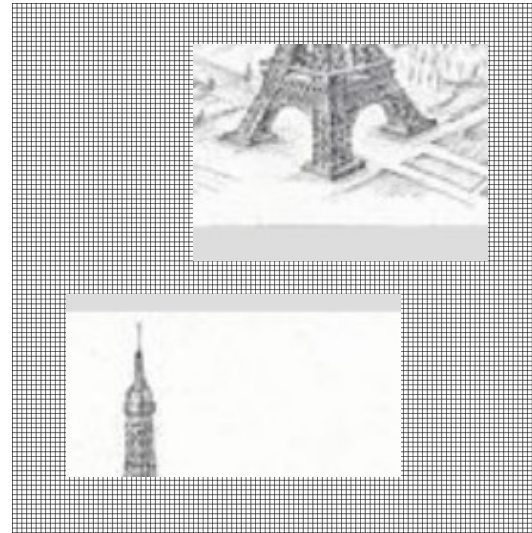
Sistemas de Referências

Efeito colateral



Sistemas de Referências

- Janelas (desde que respeitados os limites do SRU), podem ser mapeadas em qualquer posição e com diferentes tamanhos no SRD.



Parâmetros de resolução

(associados ao SRD)

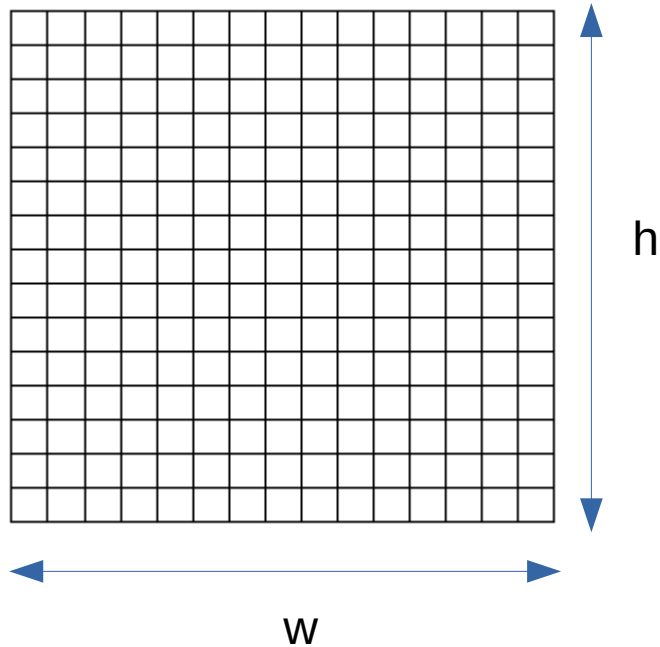
Resolução Gráfica

Vamos considerar 4 parâmetros:

- 1) ndh - o número de posições (entradas em uma matriz) endereçáveis horizontalmente;
- 2) ndv - o número de posições (entradas em uma matriz) endereçáveis verticalmente;
- 3) w - a largura do retângulo de visualização em mm ;
- 4) h - a altura do retângulo de visualização em mm .

Parâmetros de resolução

(associados ao SRD)



Assuma uma matriz com ndv linhas e ndh colunas

Parâmetros de resolução

(associados ao SRD)

Definiremos

1) Resolução horizontal:

$$\text{horiz_res} = ndh / w;$$

2) Tamanho das entradas na horizontal:

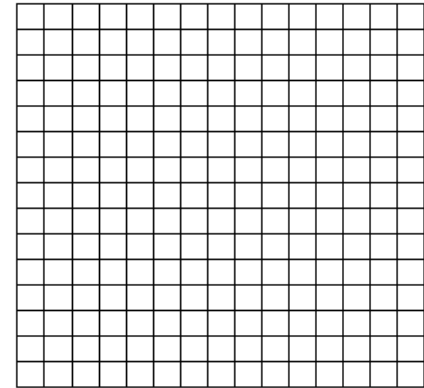
$$\text{horiz_dot_size} = w / ndh;$$

3) Resolução vertical:

$$\text{vert_res} = ndv / h;$$

4) Tamanho das entradas na vertical:

$$\text{vert_dot_size} = h / ndv;$$



Parâmetros de resolução

(associados ao SRD)

Definiremos

5) Total pontos endereçáveis:

$$nr_dots = ndh * ndv;$$

6) Resolução de área:

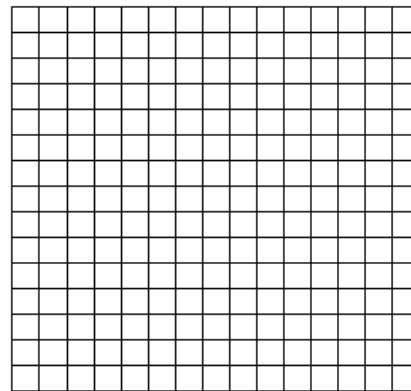
$$area_res = nr_dots / (w * h);$$

7) Razão do aspecto gráfico:

$$aspect_ratio = vert_dot_size / horiz_dot_size;$$

8) Razão do aspecto físico:

$$physical_aspect_ratio = h / w$$



Parâmetros de resolução

(associados ao SRD)

Observações

- *horiz_res*, *vert_res* e *area_res* definem resoluções físicas, enquanto que *ndh*, *ndv* e *nr_dots* definem *resoluções gráficas*;
- Dispositivos de visualização podem ter a mesma resolução gráfica com resoluções físicas muito diferentes;
- O ideal seria ter um *aspect_ratio* igual ou próximo de 1.

Representações no SRU e SRD

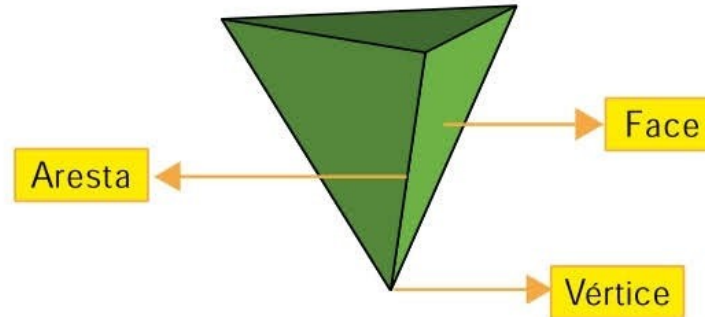
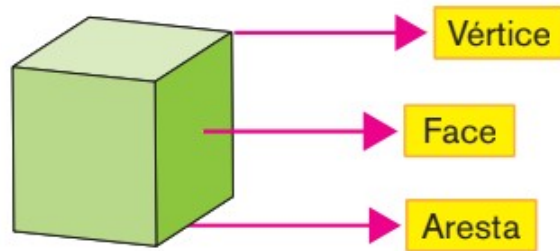
Representação de objetos

- O conceito de **objeto gráfico** é fundamental para a Computação Gráfica e áreas afins;
- Portanto, o que são e como definir objetos, faces e arestas no SRU?
- Quais são os valores que inserimos nas entradas da matriz no SRD?

Representações no SRU e SRD

Representação de objetos (SRU)

- Um objeto gráfico representa a geometria (forma) e os atributos (propriedades) de um objeto do mundo real



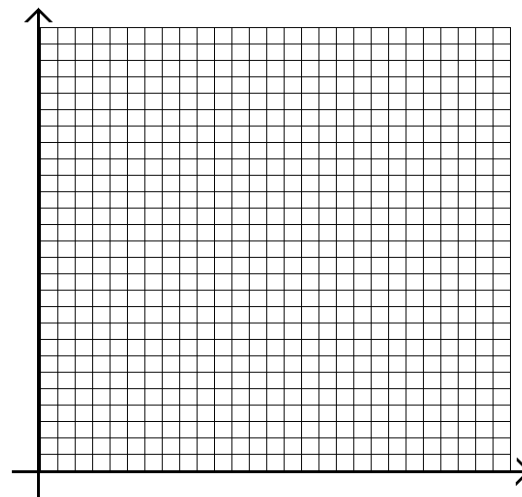
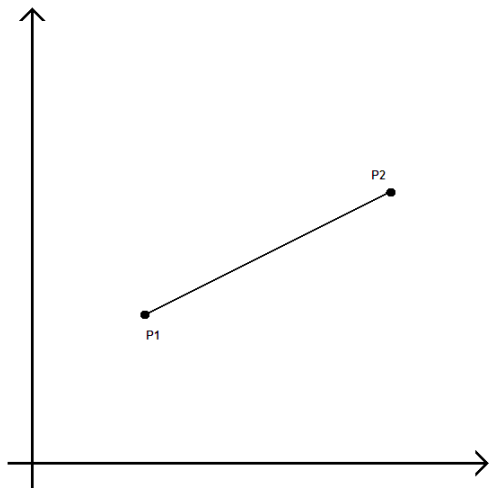
Conversão matricial

(segmentos de reta)

- O traçado de primitivas gráficas elementares, como segmentos de reta ou arcos de circunferência, requer a construção de algoritmos capazes de determinar em uma matriz de exibição quais entradas devem ser alteradas (ou preenchidas) de maneira a simular a aparência do elemento gráfico desejado;
- Reta $\rightarrow$ infinitamente fina, ou seja, área = 0;
- Problemas para apresentação em um monitor utilizando-se entradas discretas (*pixels*):
 - deve-se escolher quais entradas melhor representa um segmento de reta;
 - existem diversos algoritmos que possibilitam diferentes resultados.

Conversão matricial

(segmentos de reta)



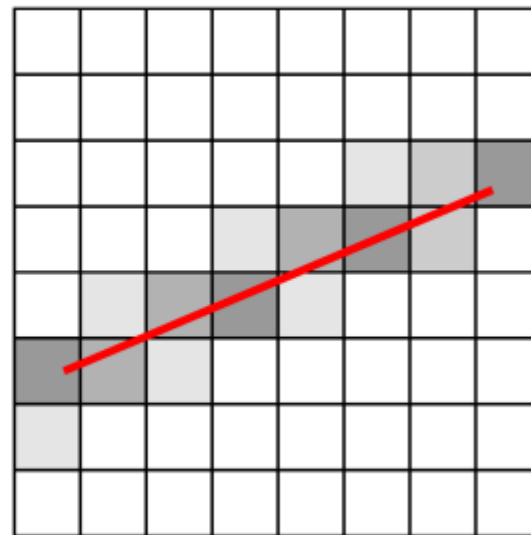
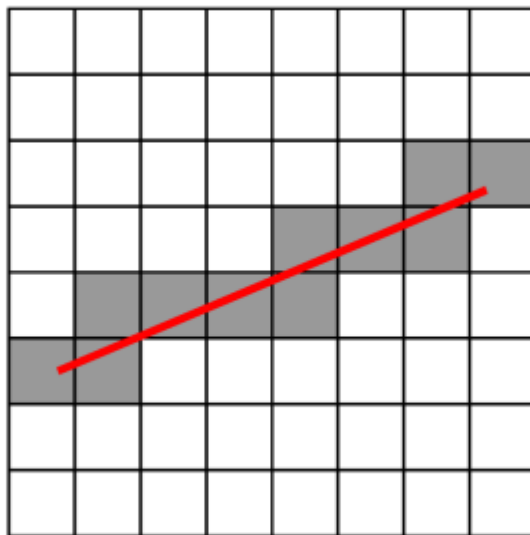
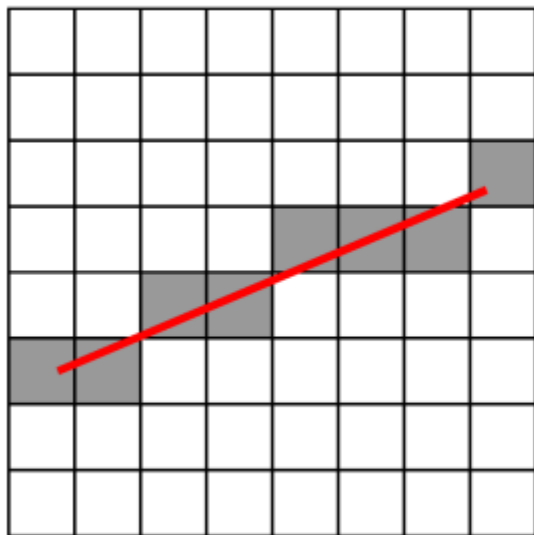
- “mundo contínuo”
representação perfeita de um
segmento de reta (ponto -
adimensional)

- Aproximação preenchendo
uma grade com entradas
discretas (possuem largura e
altura)

Conversão matricial

(segmentos de reta)

- Necessidade de **definir critérios ou convenções** sobre quais entradas devem ser preenchidas:



Conversão matricial

(segmentos de reta)

Propriedades desejadas

1. Linearidade: as entradas preenchidas devem dar a aparência de que estão sobre uma reta;
2. Precisão: os segmentos devem iniciar e terminar nas entradas especificadas;
3. Espessura (densidade) uniforme: a densidade da “linha” é dada pelo número de entradas preenchidas dividido pelo comprimento da linha;
4. Continuidade: a imagem não apresenta interrupções indesejáveis.
5. Rapidez no traçado dos segmentos.

Conversão matricial

(segmentos de reta)

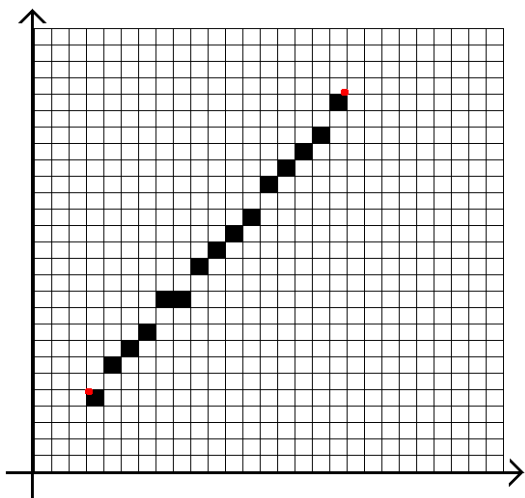
Observações de ordem prática

- Essas rotinas frequentemente são implementadas em hardware especializado (devido a complexidade computacional);
- Ou em rotinas de software escritas em linguagem de máquina otimizada;
- Entretanto, é instrutivo estudar alguns algoritmos para traçado de linhas usando uma linguagem de alto nível (proposta da disciplina).

Conversão matricial

(segmentos de reta)

- **Critério:** selecionar todas as entradas cujas coordenadas são obtidas arredondando-se os **valores das coordenadas de algum ponto do segmento**.



Proposta 1: método incremental baseado na função afim ou do 1º grau:

$$y = f(x) \rightarrow y = mx + b$$

(*Digital Differential Analyser – DDA*)

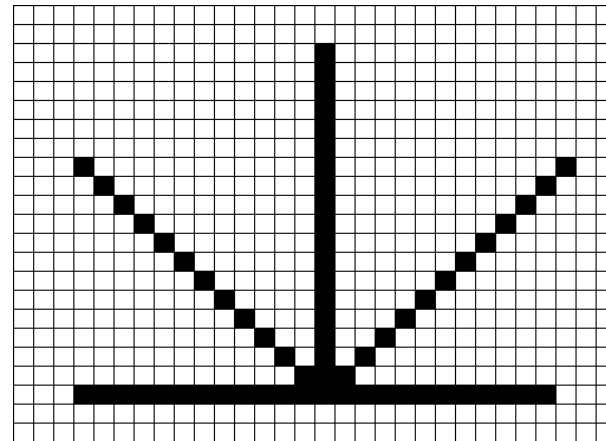
Proposta 2: algoritmo de *Bresenham*

Conversão matricial

(segmentos de reta)

Situações específicas que devem ser consideradas

- Segmentos de retas horizontais;
- Segmentos de retas verticais;
- Segmentos de reta com coeficiente angular unitário ($\pm 45^\circ$);
- Considerando que o preenchimento na vertical não é gráfico de função, a abordagem neste caso deve ser considerada separadamente.



Conversão matricial

(segmentos de reta)

Primeira proposta: Algoritmo Digital Differential Analyser

- Algoritmo incremental;
- Cria “*boas retas*”, mas consome muito tempo devido as funções de arredondamento, uma vez que trabalha com ponto flutuante;
- Com o uso da equação de reta, $y = mx + b$:

```
...
for(int x = x1; x <= x2; x++)
{
    int y = (int)round(m*x + b);
    desenharPonto(x, y, cor);
}
...
```

Conversão matricial

(segmentos de reta)

Algoritmo *Digital Differential Analyser*

- (variante) para o caso $0 \leq m \leq 1$, incrementa x de 1 ($\Delta x = 1$) e atualiza y de acordo com:

$$y_1 = mx_1 + b$$

$$x_2 = x_1 + 1$$

$$y_2 = mx_2 + b$$

$$= m(x_1 + 1) + b$$

$$= (mx_1 + b) + m$$

$$= y_1 + m$$

Conversão matricial

(segmentos de reta)

Algoritmo *Digital Differential Analyser*

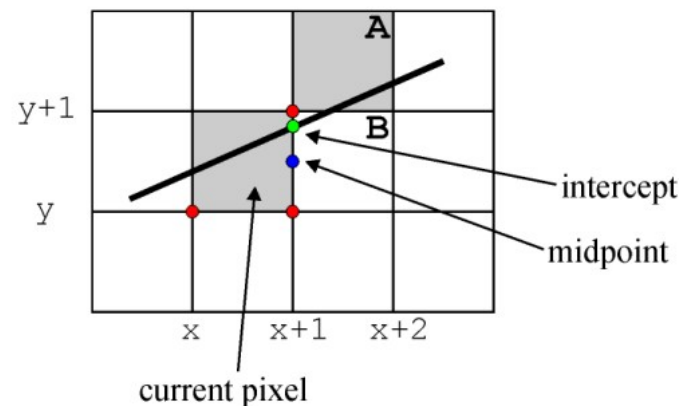
- Uma desvantagem é que o passo em x (ou y , dependendo da diferença) é igual a 1 (não existe fração das entradas da grade matricial do SRD);
- Há outros comportamentos indesejados para ângulos próximos a 0° e 90° , com a reta quase horizontal ou quase vertical;
- Como o coeficiente angular m não é necessariamente um número inteiro, obriga o hardware a trabalhar com números em ponto flutuante, consumindo tempo de execução.

Conversão matricial

(segmentos de reta)

Segunda proposta: Algoritmo de Bresenham

- Mais eficiente que o algoritmo DDA;
- Incremental em x, mas trabalha apenas com valores inteiros;
- Em cada iteração, determinamos se o segmento de reta real intersecta a próxima entrada da grade acima ou abaixo da metade do valor de y;
- Para o caso de um coeficiente angular no intervalo $0 \leq m \leq 1$, temos:
 - se acima, então: $y_{i+1} = y_i + 1$
 - caso contrário: $y_{i+1} = y_i$



Conversão matricial

(segmentos de reta)

- O **algoritmo de Bresenham** (também chamado de **algoritmo do Ponto Médio**) baseia-se no argumento de que as entradas que representam um segmento de reta devem ser vizinhas;
- Será analisado o caso de $0 \leq m \leq 1$, pois para $m > 1$, basta inverter as coordenadas x e y (simetria);
- O método consiste em determinar qual entrada será preenchida avaliando se o ponto real convertido em coordenadas do dispositivo está acima ou abaixo do segmento de reta;
- O método determina um critério baseado na função implícita da reta dada por:

$$F(x,y) = Ax + By + C$$

Conversão matricial

(segmentos de reta)

Sabemos que

- Se um ponto $p = (x,y)$ está sobre a reta, $F(x,y) = 0$;
- Se um ponto $p = (x,y)$ está abaixo da reta, $F(x,y) > 0$;
- Se um ponto $p = (x,y)$ está acima da reta, $F(x,y) < 0$;
- Considerando a equação da reta em termos de sua inclinação, aproximamos $y = f(x)$ por:

$$y = \frac{\Delta y}{\Delta x} x + b$$

- Sendo $\Delta y = y_2 - y_1$ e $\Delta x = x_2 - x_1$ (*diferenças finitas*) a partir de dois pontos dados $p_1 = (x_1, y_1)$ e $p_2 = (x_2, y_2)$, que definem o segmento de reta a ser convertido na grade (p_1 e p_2 estão sobre a reta!).

Conversão matricial

(segmentos de reta)

Portanto

- Obtemos, na forma implícita, $F(x,y) = \Delta y \cdot x - \Delta x \cdot y + \Delta x \cdot b = 0$, onde $A = \Delta y$, $B = -\Delta x$ e $C = \Delta x \cdot b$, para um ponto sobre a reta;
- O ponto de partida é a seguinte pergunta: dado um ponto inicial sobre a reta ($F(x,y) = 0$, onde $0 \leq m \leq 1$), a próxima entrada será $(x+1,y)$ ou $(x+1, y+1)$?
- Devemos estabelecer o valor da variável de decisão inicial determinando o valor de $F(x+1,y+\frac{1}{2})$;
- Esse valor dá origem ao nome do método, algoritmo do ponto médio;
- Uma parte fracionária é introduzida mas como estamos interessado apenas no sinal de uma **variável de decisão**, podemos multiplicar por 2 sem alterar o resultado.

Conversão matricial

(segmentos de reta)

Para o valor de decisão inicial

$$2 \cdot F(x+1, y+\frac{1}{2}) = 2 \cdot \{\Delta y \cdot (x+1) - \Delta x \cdot (y+\frac{1}{2}) + \Delta x \cdot b\} \rightarrow$$

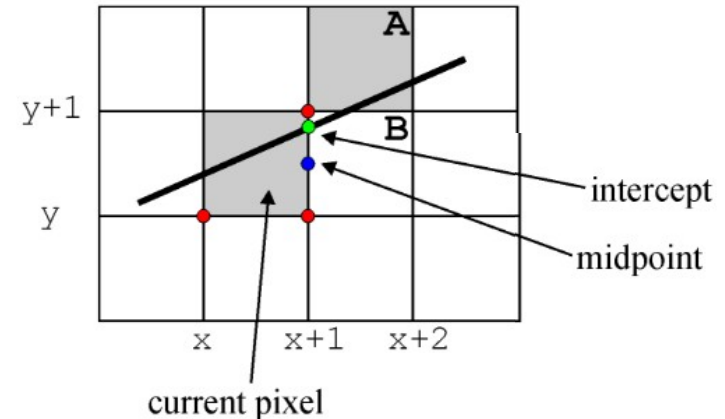
$$2 \cdot F(x+1, y+\frac{1}{2}) = 2 \cdot \Delta y \cdot x + 2 \cdot \Delta y - 2 \cdot \Delta x \cdot y - \Delta x + 2 \cdot \Delta x \cdot b \rightarrow$$

$$2 \cdot F(x+1, y+\frac{1}{2}) = 2 \cdot \Delta y \cdot x - 2 \cdot \Delta x \cdot y + 2 \cdot \Delta x \cdot b + 2 \cdot \Delta y - \Delta x \rightarrow$$

$$2 \cdot F(x+1, y+\frac{1}{2}) = 2 \cdot F(x, y) + 2 \cdot \Delta y - \Delta x \rightarrow$$

$$2 \cdot F(x+1, y+\frac{1}{2}) = 2 \cdot \Delta y - \Delta x,$$

uma vez que $F(x, y) = 0$.



Conversão matricial

(segmentos de reta)

Considerando o sinal da variável de decisão, temos

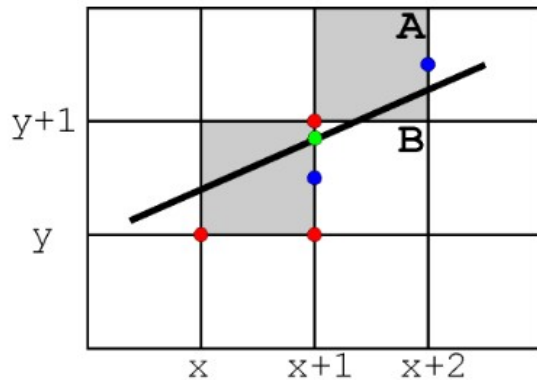
- O valor da variável de decisão inicial será dado por $e = 2 \cdot \Delta y - \Delta x$;
- Do exposto acima, se $e > 0$, a próxima entrada a ser preenchida será $(x+1, y+1)$, pois o “ponto médio está abaixo da reta”;
- Se $e < 0$, a próxima entrada a ser preenchida será $(x+1, y)$, pois o “ponto médio está acima da reta”;
- Os próximos “desvios” são calculados considerando $2 \cdot F(x+1, y+1)$ e $2 \cdot F(x+1, y)$;
- Se $e \geq 0$, modifica-se “ $e = e + 2 \cdot \Delta y - 2 \cdot \Delta x$ ” e a próxima entrada a ser preenchida será $(x+1, y+1)$, pois o “ponto médio está acima da reta”;
- Se $e < 0$, modifica-se “ $e = e + 2 \cdot \Delta y$ ” e a próxima entrada a ser preenchida será $(x+1, y)$, pois o “ponto médio está abaixo da reta”.

Conversão matricial

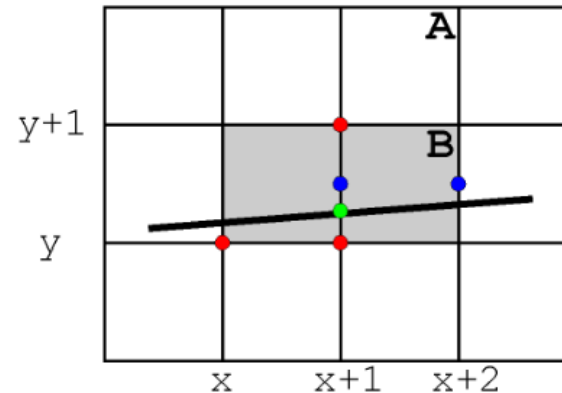
(segmentos de reta)

Concluindo, temos

- Nos próximos passos, se $e \geq 0$, modifica-se " $e = e + 2 \cdot \Delta y - 2 \cdot \Delta x$ " e a próxima entrada a ser preenchida será $(x+1, y+1)$;
- Se $e < 0$, modifica-se " $e = e + 2 \cdot \Delta y$ " e a próxima entrada a ser preenchida será $(x+1, y)$.



$$e \geq 0$$



$$e < 0$$

Conversão matricial

(segmentos de reta)

Observações

- Foi analisado o caso para $0 \leq m \leq 1$, pois para $m > 1$, basta inverter as coordenadas x e y (simetria em relação a reta com 45° em relação ao eixo x);
- Da mesma forma, para $-1 \leq m < 0$, analogamente, determinamos os valores dos desvios considerando as equações:
 - $2 \cdot F(x+1, y+\frac{1}{2})$ para o desvio inicial; e
 - $2 \cdot F(x+1, y)$ e $2 \cdot F(x+1, y-1)$ para considerar o preenchimento das entradas $(x+1, y)$ ou $(x+1, y-1)$, respectivamente;
- O caso $m < -1$ também é obtido por simetria em torno da reta -45° em relação ao eixo x .

Conversão matricial

(segmentos de reta)

```
...
dx = x2 - x1;
dy = y2 - y1;
d = 2*dy - dx;
x = x1;
y = y1;
desenharPonto(x, y, cor);
while (x < x2) {
    if (d <= 0) {
        d = d + (2*dy);
    }
    else {
        d = d + 2*(dy - dx);
        y = y + 1;
    }
    x = x + 1;
    desenharPonto(x, y, cor);
}
```

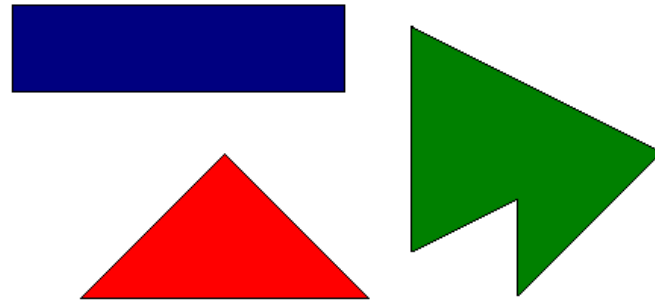
} inicialização

} seleciona B

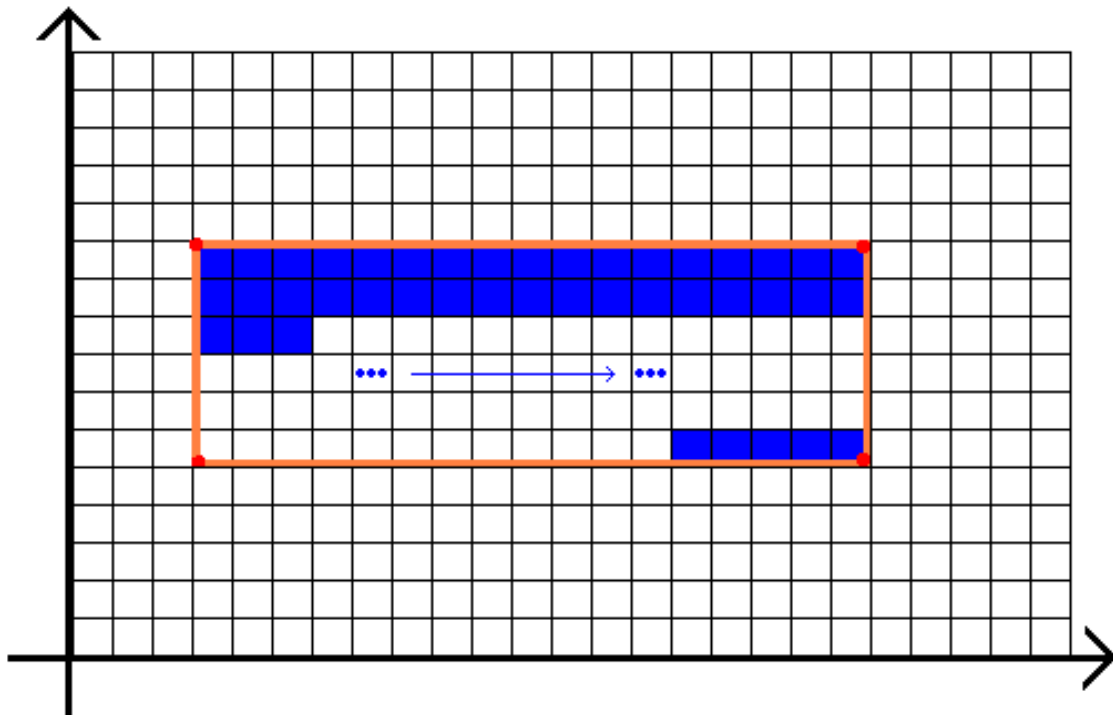
} seleciona A

Preenchimento de regiões

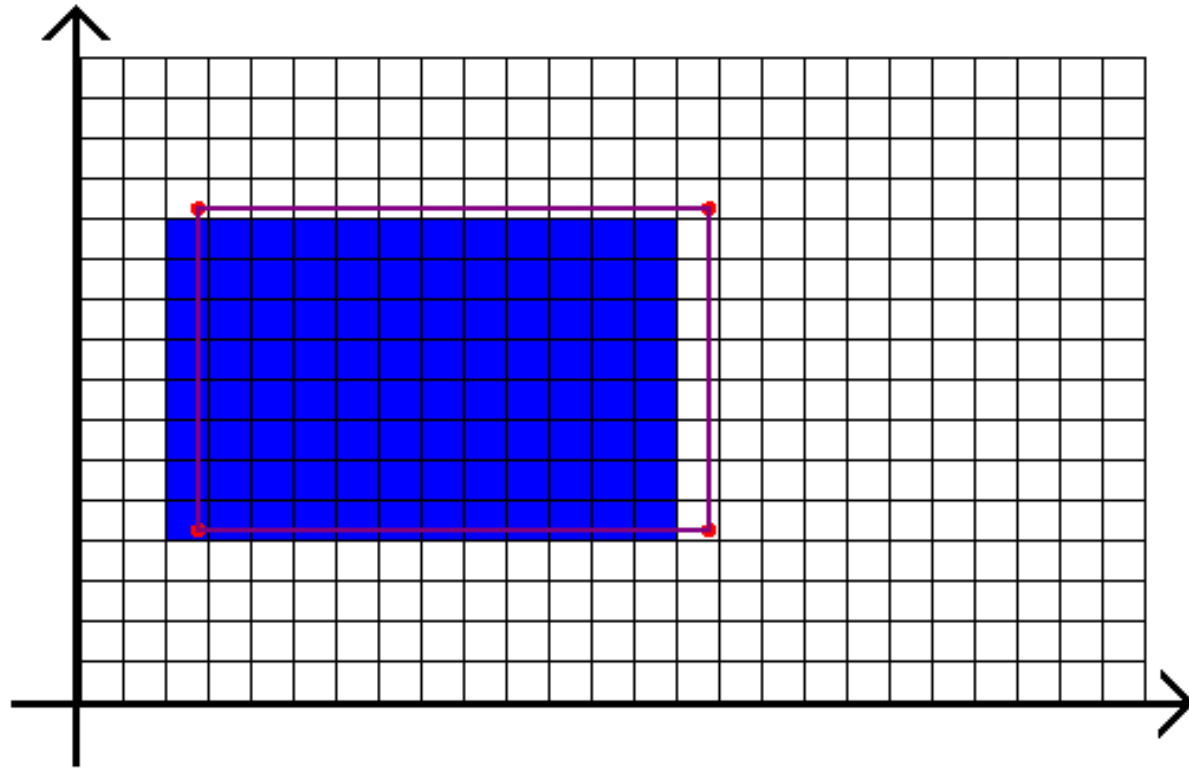
- A tarefa de preencher um polígono pode ser dividida em duas etapas:
 - 1) Decidir que entradas pintar para preencher o polígono; e
 - 2) Decidir com qual valor pintar a entrada.



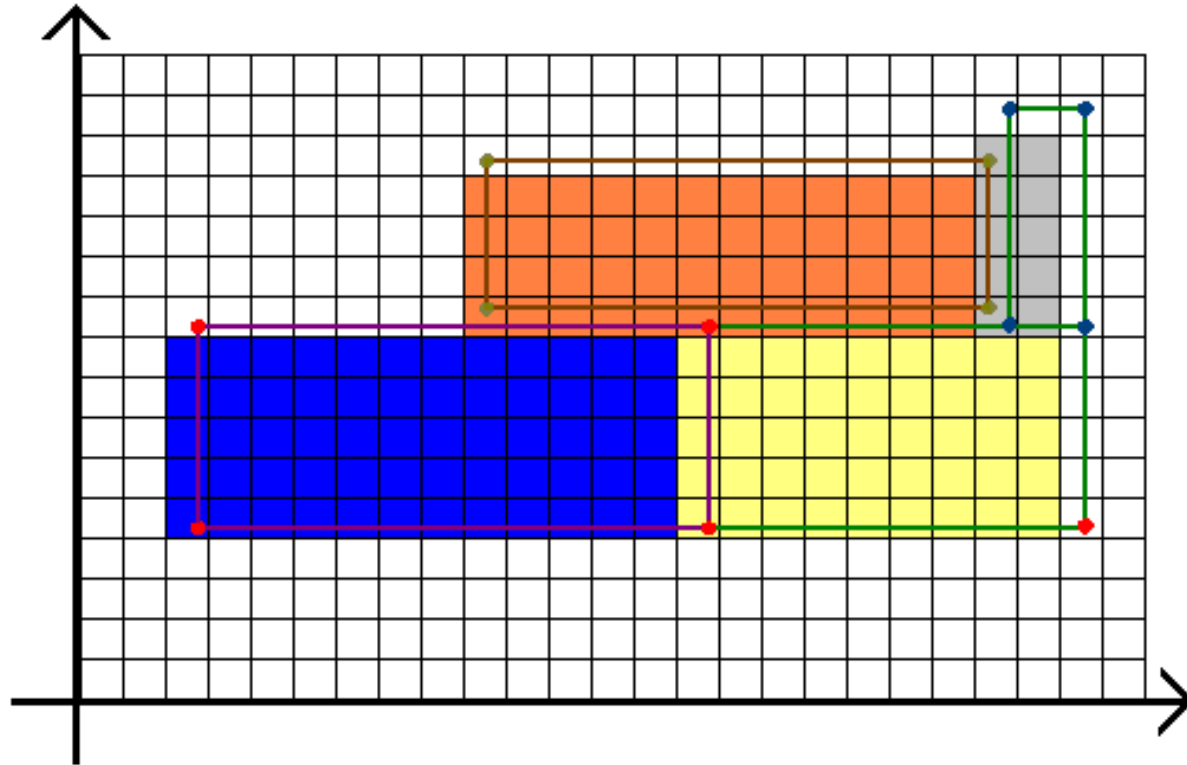
Preenchimento de regiões



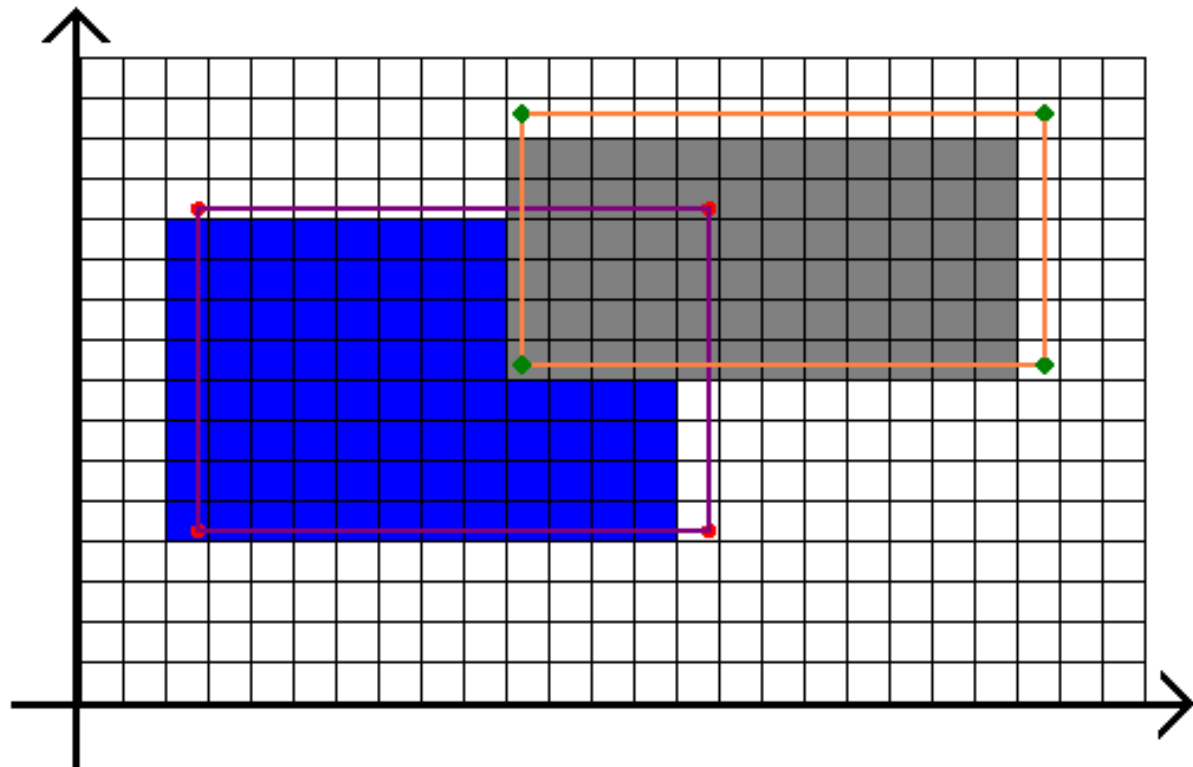
Preenchimento de regiões



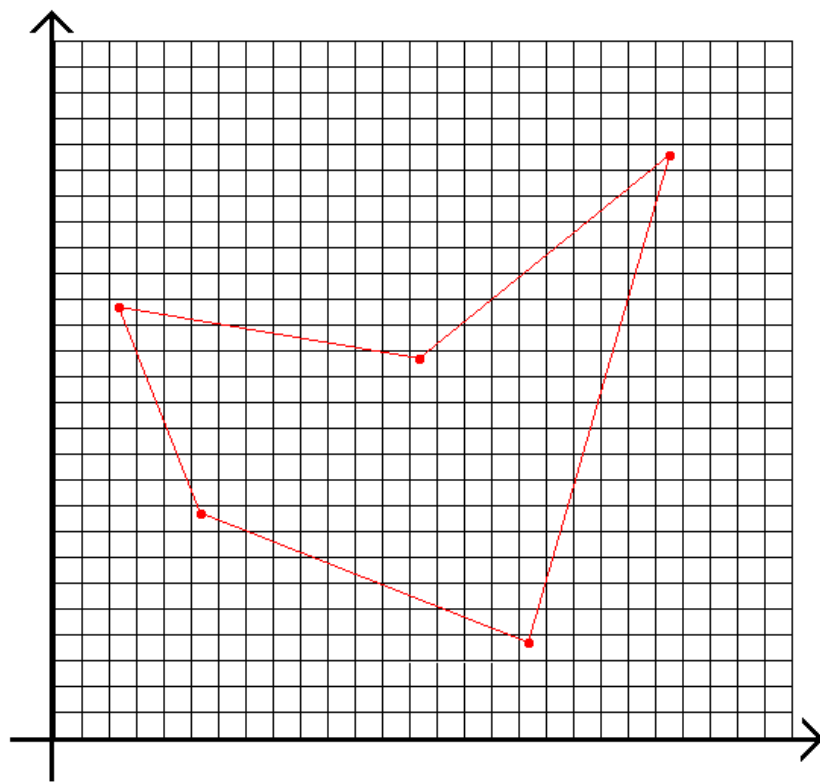
Preenchimento de regiões



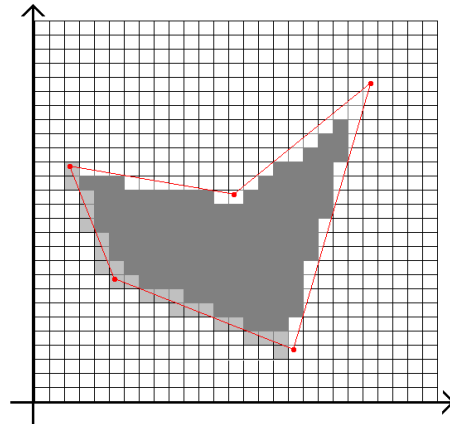
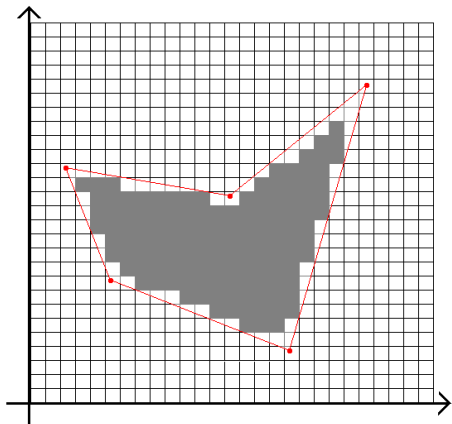
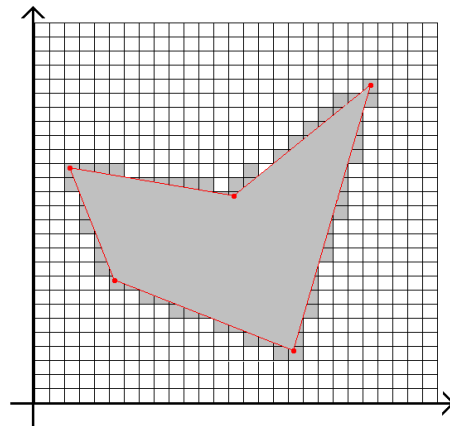
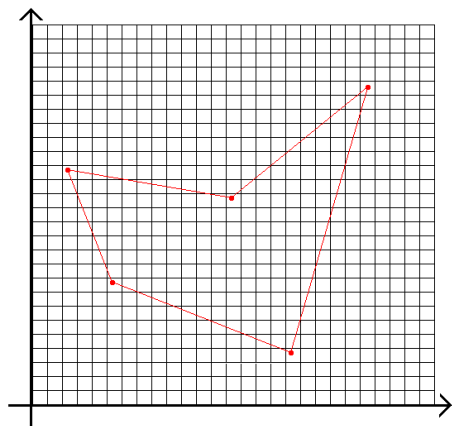
Preenchimento de regiões



Preenchimento de regiões



Preenchimento de regiões



Operações sobre Objetos

Transformações geométricas no plano

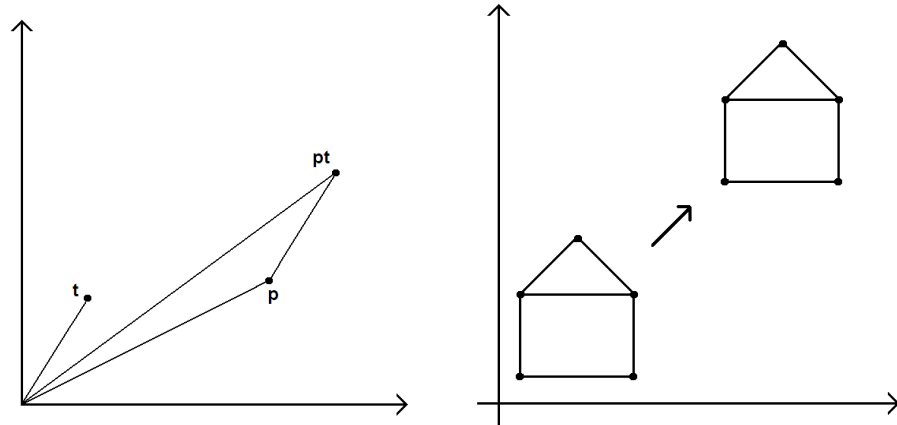
Todas as transformações geométricas sobre objetos em CG podem ser realizadas através de transformações (operadores) lineares. Dentre as mais usuais, destacamos:

- Deslocamento ou translação;
- Escalonamento;
- Rotação;
- Espelhamento.

Operação de Translação

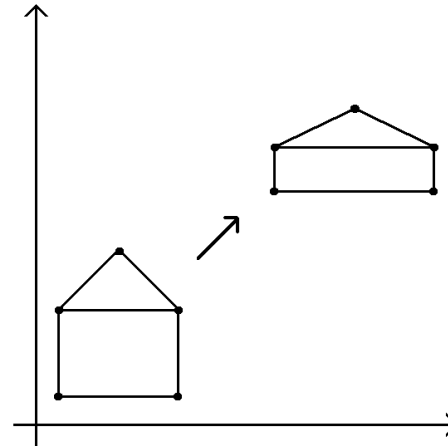
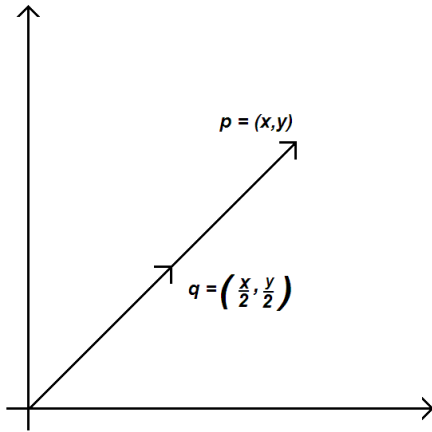
- Transladar significa movimentar um objeto;
- Transladamos um objeto quando transladamos todos os seus pontos;
- Dado um ponto $p = (x, y)$, podemos transladá-lo somando a este ponto outro vetor de coordenadas $d = (dx, dy)$, obtendo como resultado o ponto $q = (x+dx, y+dy)$.

$$\begin{bmatrix} x+dx \\ y+dy \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} dx \\ dy \end{bmatrix}$$



Operação de Escalonamento

- Escalonar significa mudar as dimensões de escala de um vetor;
- Escalonamos um objeto no plano quando escalonamos todos os seus pontos;
- Dado um ponto $p = (x, y)$, podemos escaloná-lo multiplicando cada coordenada deste por constantes S_x e S_y , $S = (S_x, S_y)$, obtendo como resultado $pe = (x*S_x, y*S_y)$.



Operação de Rotação

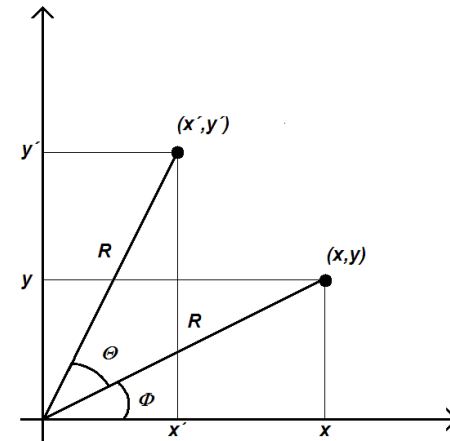
- Rotacionar um ponto significa girar o vetor em torno da origem;
- Rotacionamos um objeto no plano quando fazemos esta operação para todos os seus pontos;
- Se um ponto $p = (x,y)$, a uma distância R da origem do sistema de coordenadas, for rotacionado de um ângulo Θ , em torno da origem, *no sentido anti-horário*, suas novas coordenadas serão descritas por $q = (x', y')$, onde

$$x' = R \cos(\Theta + \Phi) \text{ e } y' = R \sin(\Theta + \Phi)$$

- Equivalentemente, as coordenadas de q são dadas por:

$$x' = x \cos(\Theta) - y \sin(\Theta)$$

$$y' = y \cos(\Theta) + x \sin(\Theta)$$



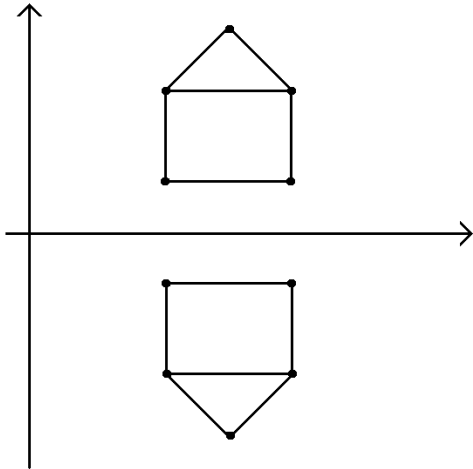
Operações de Reflexão

- A transformação de reflexão em torno de um eixo, ou espelhamento (*flip*), aplicada a um objeto, produz um novo objeto que é como se este fosse visto reproduzido por um espelho, posicionado no eixo em torno do qual se faz a reflexão;
- Podemos considerar três tipos de reflexões:
 - Em torno do eixo x ;
 - Em torno do eixo y ;
 - Em torno dos dois eixos, x e y .

Operações de Reflexão

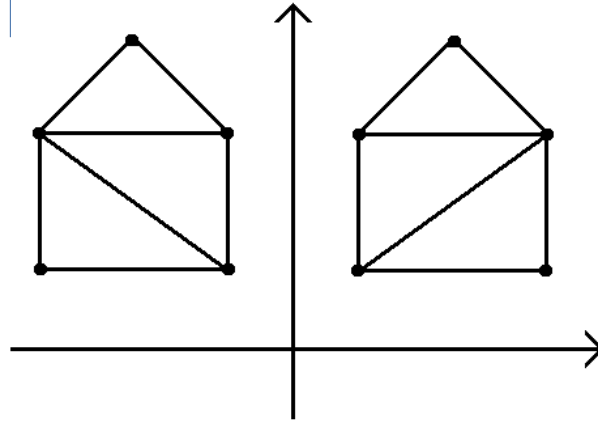
*Reflexão em
torno do eixo x:*

Dado $p = (x,y)$, o ponto q refletido em torno do eixo x é dado por $q = (x,-y)$.



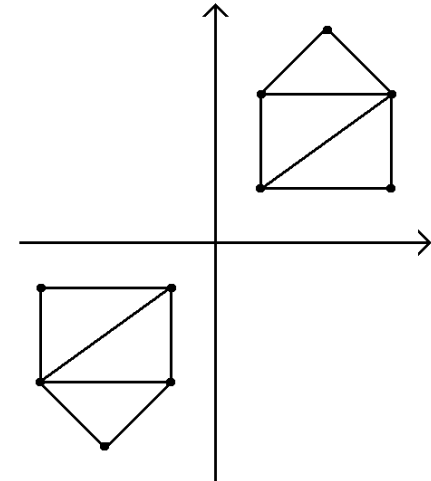
*Reflexão em torno
do eixo y:*

Dado $p = (x,y)$, o ponto q refletido em torno do eixo y é dado por $q = (-x,y)$.



*Reflexão em torno
do dois eixos:*

Dado $p = (x,y)$, o ponto q refletido em torno do eixo x e do eixo y é dado por $q = (-x,-y)$.



Coordenadas Homogêneas

- Com a introdução do conceito de *coordenadas homogêneas*, todas as transformações podem ser tratadas como operadores lineares, ou seja, através da multiplicação de uma matriz por um vetor de coordenadas;
- Em coordenadas homogêneas, adicionamos uma terceira coordenada ao ponto. Assim, ao invés do ponto p ser representado pelas coordenadas (x,y) , este passa a ser representado por uma tripla ordenada (x,y,w) ;
- A componente w é artificial e a representação continua sendo de um ponto no plano cartesiano;
- A componente homogênea w necessariamente **deve ser diferente de zero**.

Coordenadas Homogêneas

- Dizemos que dois pontos em coordenadas homogêneas representam o mesmo ponto no espaço bidimensional se e somente se um é múltiplo de outro;
- Por exemplo, $(2, 3, 6)$ e $(4, 6, 12)$ são triplas ordenadas que representam o mesmo ponto;
- Portanto, podemos dizer que um ponto do plano possui infinitas representações homogêneas;
- Se w é uma coordenada diferente de zero, podemos dividir (x, y, w) por w , obtendo $(x/w, y/w, 1)$;
- As coordenadas x/w e y/w são chamadas de *coordenadas cartesianas do ponto homogêneo*.

Coordenadas Homogêneas

- Embora w possa assumir qualquer valor, usualmente, assumimos $w = 1$;
- Portanto, podemos obter pontos homogeneizados, assumindo o plano $w = 1$, da forma $(x, y, 1)$;
- Podemos agora escrever todas as operações como **translações**, **rotações** e **escalonamentos** (assim como as operações de **reflexão**) como operadores lineares no espaço tridimensional.

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & d_x \\ 0 & 1 & d_y \\ 0 & 0 & 1 \end{bmatrix}$$

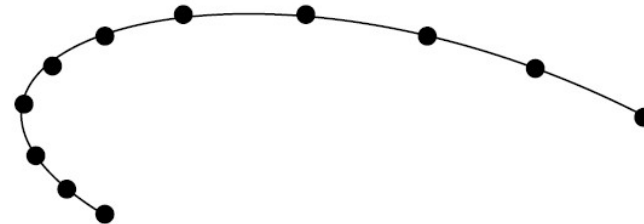
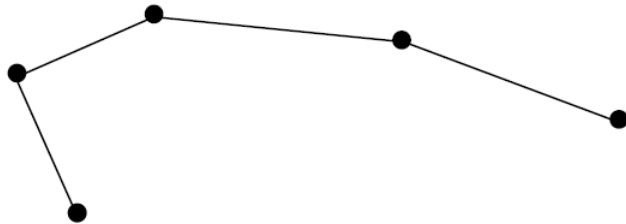
Representações de Curvas

- Existem duas formas para representação de curvas (e superfícies):
 - Conjunto de pontos;
 - Representação analítica.
- As abordagens analíticas podem ser valiosas utilizando as mesmas técnicas para as conversões matriciais de segmentos de retas, principalmente com a metodologia do ponto médio, uma vez que conhecemos a equação da curva desejada;
- Como exemplo, cônicas e quádricas, podem ser obtidas determinando-se as próximas entradas de maneira similar ao *algoritmo de Bresenham* para segmentos de retas.

Representações de Curvas

Conjunto de pontos

- Nesse caso, uma representação visual adequada pode ser gerada conectando-se os pontos por segmentos de reta, *se estes forem espaçados adequadamente.*



Representações de Curvas

Conjunto de pontos

- Observe que nas regiões da curva onde a curvatura é grande, a aproximação por segmentos de reta é particularmente ruim;
- Uma maneira de melhorar a representação é aumentar a densidade dos pontos nessas regiões;
- A representação de formas complexas bidimensionais ou tridimensionais, utilizando-se simplesmente de elementos lineares, como segmentos de reta ou poliedros de faces planas, pode exigir uma quantidade enorme desses elementos para se atingir um grau de realismo satisfatório.

Representações de Curvas

Representação analítica

- Nesse caso, a representação é feita através de *formulações matemáticas* (que podem ser paramétricas ou não-paramétricas);
- Particularmente, as equações não-paramétricas podem ser implícitas ou explícitas;
- A grande vantagem das formas implícitas é que com elas é muito fácil verificar se um dado ponto pertence ou não a curva (*compare com o algoritmo de Bresenham*);
- Vale observar que a abordagem implícita é mais utilizada na representação bidimensional, uma vez que o processo tridimensional é lento;
- A desvantagem no uso de equações não-paramétricas é que são dependentes do sistema de coordenadas utilizado (a escolha afeta a facilidade de uso).

Representações de Curvas

Representação analítica

- Por outro lado, as paramétricas possuem vantagens como:
 - ✓ Precisão;
 - ✓ armazenagem compacta;
 - ✓ facilidade de cálculo (exato) de pontos intermediários (em uma representação por pontos, outros intermediários precisam ser determinados por interpolação);
 - ✓ facilidade para calcular propriedades da curva como inclinação e curvatura;
 - ✓ facilidade para desenhar as curvas;
 - ✓ facilidade para fazer alterações contínuas no formato da curva de forma a atender requisitos de projeto.

Representações de Curvas

Propriedades desejadas na representação de curvas

- ✓ Independência do sistema de coordenadas;
- ✓ Possibilidade de se efetuar um controle local;
- ✓ Versatilidade;
- ✓ Continuidade e diferenciabilidade da curva gerada;
- ✓ Redução das oscilações;
- ✓ Facilidade de introdução de parâmetros de controle.

Representações de Curvas

- Considerando as características acima, estudaremos duas representações paramétricas muito utilizada em CG;
- Frequentemente utilizamos representações de terceira ordem:
 - ✓ Curvas de *Bézier*;
 - ✓ Curvas *B-Splines*;
- Frequentemente, essas abordagens são utilizadas para interpolação entre pontos, mas são ferramentas úteis na geração de curvas com formas arbitrárias.

Representações de Curvas

Curvas de Bézier

A curva paramétrica de Bézier é definida pelo polinômio (em t)

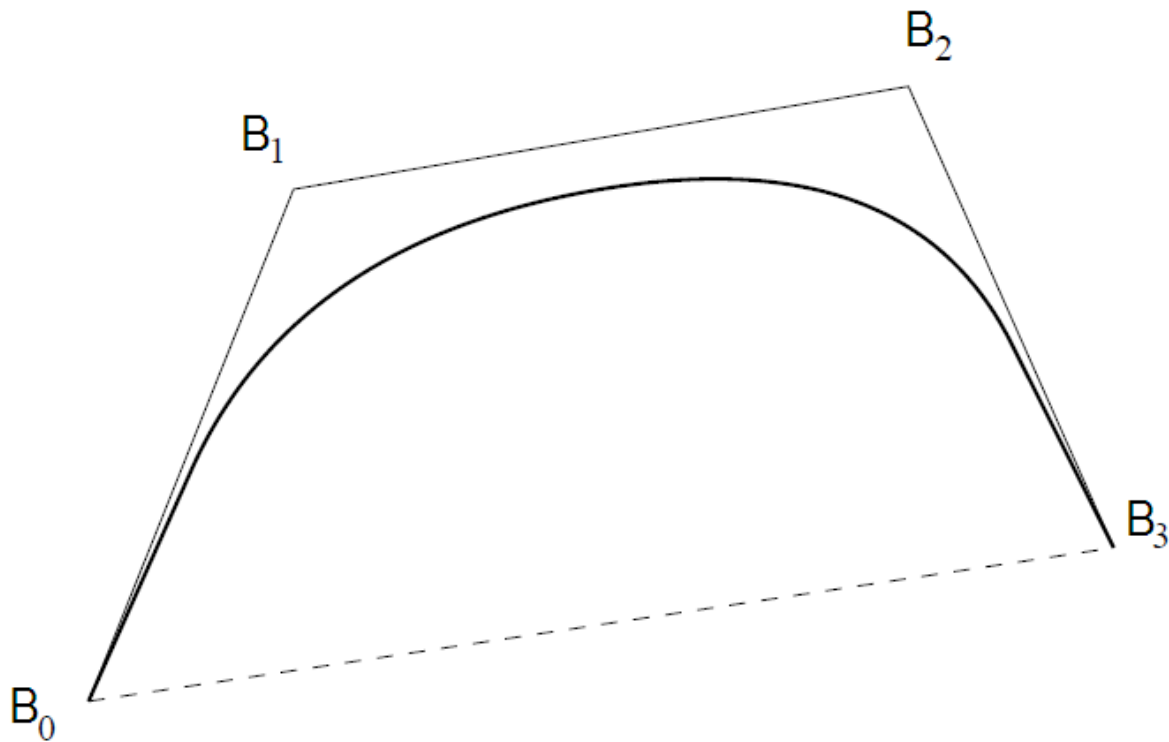
$$P(t) = \sum_{i=0}^n B_i J_{n,i}(t), \quad 0 \leq t \leq 1$$

onde B_i são os $n+1$ pontos de controle e $J_{n,i}(t)$ são funções de base dadas por:

$$J_{n,i}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

Representações de Curvas

Fecho convexo



Representações de Curvas

As Curvas de Bézier satisfazem as propriedades

- $P(t)$ é um polinômio de grau n ;
- Os vetores tangentes no primeiro e último pontos têm a mesma direção que o primeiro e últimos segmentos do polígono de controle, respectivamente;
- O primeiro e último ponto de controle pertencem a curva;
- A curva está contida no fecho convexo do polígono de controle.

Representações de Curvas

Curvas *B-Splines*

As funções de base para as *curvas B-Spline* podem ser definidas de maneira recursiva como:

$$N_{i,1}(t) = \begin{cases} 1 & \text{para } t_i \leq t \leq t_{i+1} \\ 0 & \text{nos demais intervalos} \end{cases}$$

$$N_{i,k}(t) = \left(\frac{1 - t_i}{t_{i+k-1} - t_i} \right) N_{i,k-1}(t) + \left(\frac{t_{i+k} - t}{t_{i+k} - t_{i+1}} \right) N_{i+1,k-1}(t)$$

Representações de Curvas

Curvas *B-Splines*

- Essa formulação requer a escolha de um conjunto de valores t_i chamados nós, que se relacionam ao parâmetro t ;
- O parâmetro k controla a ordem de continuidade da curva, e n o número de pontos de controle usados;
- $N_{i,k}(t)$ representam as funções de grau $(k-1)$ (ordem do polinômio);
- $N_{i,k}(t)$ são curvas de continuidade C_{k-2} .

Representações de Curvas

Propriedades

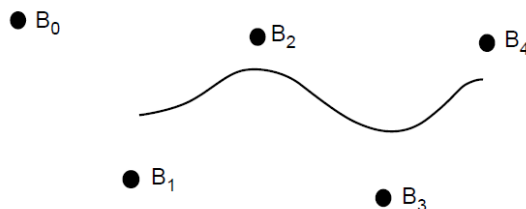
- A curva pode ser gerada para qualquer número de pontos de controle e grau de polinômio, ou seja, o grau do polinômio pode ser selecionado de maneira independente do número de pontos de controle;
- As curvas possuem controle local, isto é, as alterações nos pontos de controle da *B-Spline* apenas se propagam para os vizinhos mais próximos;
- As curvas *B-Splines* satisfazem a propriedade de envoltória convexa;
- Propriedade normalizante:

$$\sum_{i=0}^n N_{i,k}(t) = 1$$

Representações de Curvas

Característica

- A função *B-Spline* não passa pelos pontos de controle.



Restrições

- Os nós devem estar em ordem não decrescente, ou seja, os valores dos elementos t_i devem satisfazer a relação $t_i \leq t_{i+1}$;
- Um mesmo valor não deve aparecer mais que k vezes, ou seja, não pode surgir mais vezes que a ordem da *Spline* usada.

Representação de Cores

Conceitos

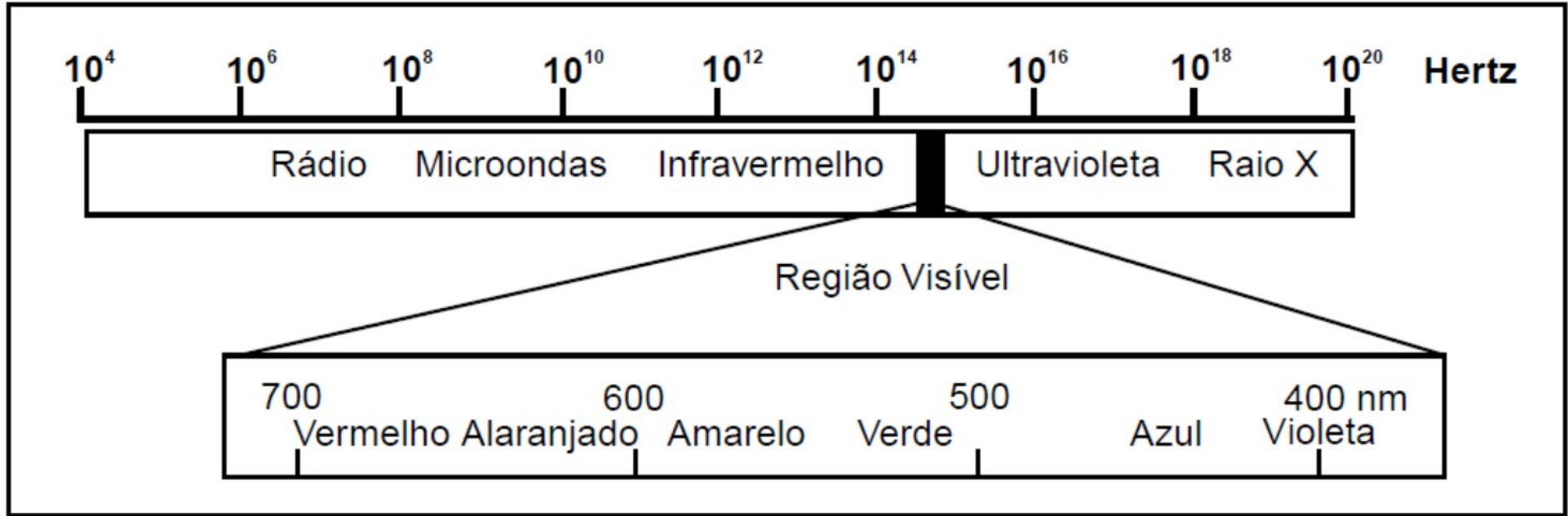
- A luz é uma onda eletromagnética que permite ver o mundo ao redor;
- O que vemos como cores em uma luz são, na verdade, diferenças de comprimento de onda.

Espectro Visível

Violeta	Azul	Ciano	Verde	Amarelo	Laranja	Vermelho
380-450	540-480	480-490	490-560	560-580	580-600	600-700

Comprimentos de onda em nanômetros

Representação de Cores

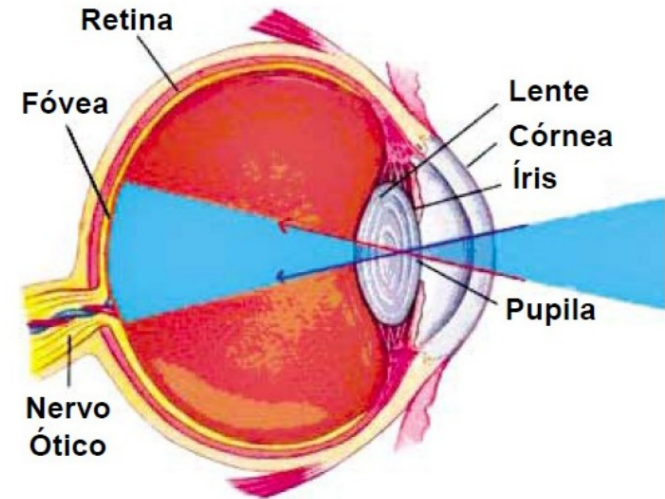


Representação de Cores

- A cor, elemento fundamental em qualquer processo de comunicação, merece uma atenção especial;
- É um componente com grande influência no dia a dia de uma pessoa, interferindo nos sentidos, emoções e intelecto;
- O uso apropriado de cores pode resultar em uma rápida e correta assimilação da informação.
- Particularmente, o uso de cores em imagens permite:
 - Mostrar as coisas conforme são vistas na natureza;
 - Representar associações simbólicas;
 - Chamar e direcionar a atenção;
 - Enfatizar alguns aspectos sociais;
 - Determinar um estado de espírito;
 - Tornar uma imagem mais fácil de ser memorizada.

O Sistema Visual Humano

- As diferentes sensações que uma luz pura produz no olho humano, e que produzem as cores, dependem da frequência ou comprimento de onda dessa luz;
- O olho humano é sensível à radiação eletromagnética na faixa de 400 a 700 nanômetros, chamada espectro visível, dentro da qual estão localizadas as chamadas sete cores visíveis, distinguidas por seus respectivos comprimentos de onda.



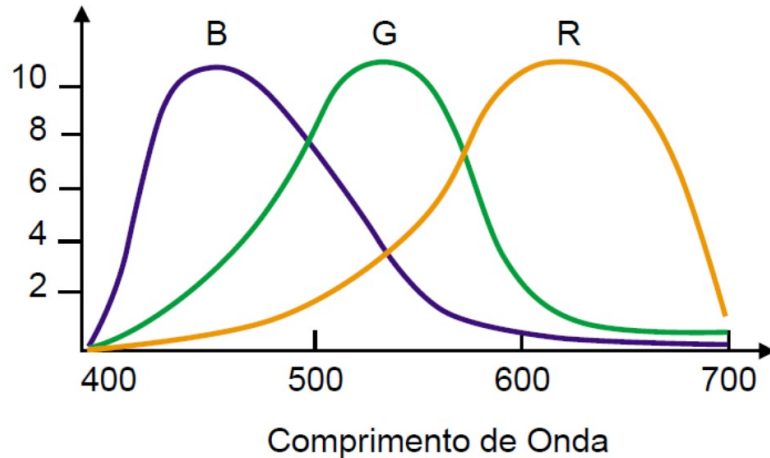
O Sistema Visual Humano

- A retina contém dois tipos de células, chamadas bastonetes e cones.
- Os bastonetes cuidam da visão em condições com pouca luz, e os cones são os responsáveis pela visão de cores e detalhes.
- Quando a luz entra em contato com estes dois tipos de células, ocorre uma série de reações químicas complexas.



O Sistema Visual Humano

Resposta Relativa



- Particularmente, existem cones que possuem respostas diferentes para comprimentos de ondas específicos;
- A figura ilustra a resposta para três comprimentos de onda relativos as faixas do espectro visível que correspondem ao vermelho, verde e azul.

Modelos de Representação

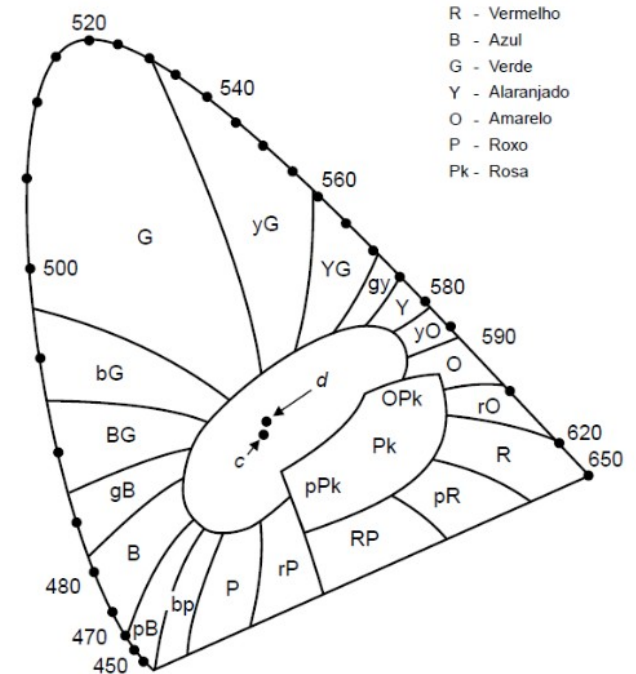
As cores primárias

- Cores primárias são cores básicas que podem ser usadas para produzir outras cores;
- Outras cores podem ser produzidas a partir de uma combinação das primárias, ou então, da composição de suas combinações;
- Não existe um conjunto finito de cores primárias visíveis que produza realmente todas as cores da natureza, mas sabe-se que uma grande parte delas pode ser reproduzida com precisão a partir de três primárias escolhidas das extremidades e centro do espectro de luzes visíveis.

Modelos de Representação

As cores primárias

- Para definir todas as cores visíveis através de radiações primárias, a *Commission Internationale de l'Éclairage* – CIE (Comissão Internacional de Iluminação – <https://cie.co.at/>) definiu três primárias (que não possuem correspondentes físicos) que podem ser combinadas para formarem todas as cores;
- Dessa forma, a CIE estabeleceu um padrão para representar as cores visíveis, denominado XYZ, formado por três coeficientes que representam as porcentagens de vermelho, verde e azul existentes em uma cor;
- A introdução de uma representação matemática no processo de especificação de cor gera muitos benefícios já que permite a especificação de uma cor através de um sistema de coordenadas geralmente cartesiano.



Modelos de Representação

O espaço de cores

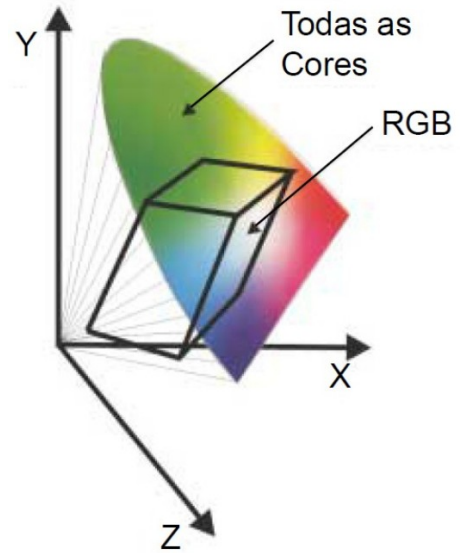
- O universo de cores que podem ser reproduzidas por um sistema é chamado de espaço de cores (*gamut*);
- Um espaço de cor é um método formal, necessário para se quantificar as sensações visuais das cores, que podem assim ser precisamente especificadas;
- Os sistemas de cores podem ser:
 - Aditivos (luz), como por exemplo, o modelo RGB;
 - Subtrativos (pigmentos), como por exemplo, o modelo CMY.

Modelos de Representação

O modelo RGB

- O modelo aditivo RGB possui como primárias as cores aditivas *vermelha* (R), *verde* (G) e *azul* (B);
- Este modelo baseia-se na sensibilidade do olho e usa um sistema de coordenadas cartesianas R, G, B, cujo subespaço de interesse é o cubo unitário;
- Em uma imagem colorida, a representação da cor C , de cada pixel da imagem, pode ser obtida matematicamente por:

$$C = rR + gG + bB$$



Modelos de Representação

O modelo aditivo RGB

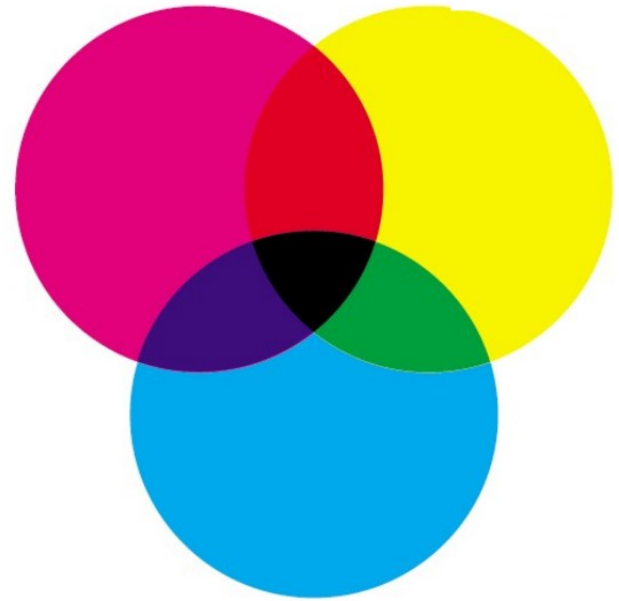


A partir da ausência de luz (representado como preto), as cores primárias, *vermelho*, *azul* e *verde* são utilizadas (em diferentes proporções) para compor todas as demais cores até chegar na cor branca (que representa a soma das três primárias)

Modelos de Representação

O modelo subtrativo CMY

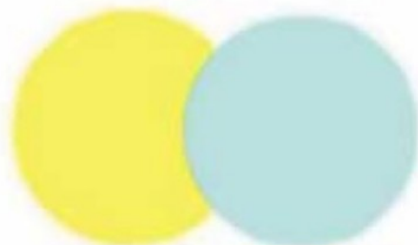
- Em uma imagem colorida, para objetos que não emitem luz própria, podemos utilizar as cores *Ciano*, *Magenta* e *Amarelo* como as primárias na formação de outras cores:
- **Ciano**: se absorve a componente vermelha da luz branca refletida. A luz branca é a soma das cores azul, verde e vermelho. Assim, em termos de cores aditivas, ciano é a soma de verde e azul;
- **Magenta**: se retira a componente verde da luz branca, sendo assim, a soma das cores aditivas vermelho e azul;
- **Amarelo**: se subtrai a componente azul da luz branca refletida. É a soma das cores aditivas verde e vermelho.



Modelos de Representação

O modelo subtrativo CMY

Amarelo
+
Ciano



=



Verde

Magenta
+
Amarelo



=



Vermelho

Ciano
+
Magenta



=

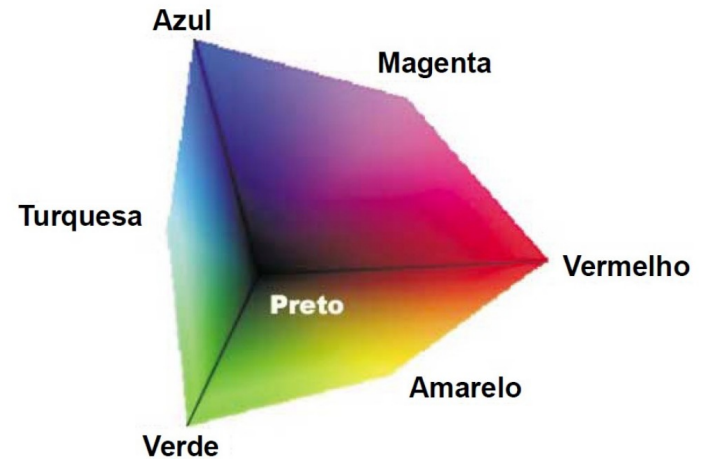


Azul

Modelos de Representação

Modelos complementares

- Os modelos aditivo RGB e subtrativo CMY são ditos complementares;
- No modelo RGB, a cor preta é representada pela ausência das componentes vermelha, azul e verde e o branco é representado pela presença das três componentes;
- No modelo CMY, a cor preta é representada pela presença das três componentes primárias do modelo RGB e a cor branca pela ausência destas.

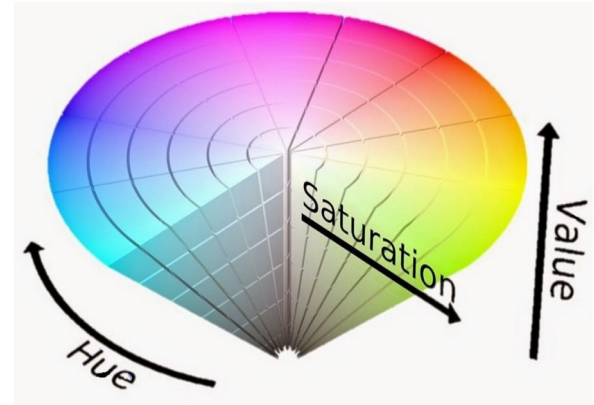


Cubo unitário representando
os modelos complementares
RGB e CMY

Modelos de Representação

O Modelo HSV

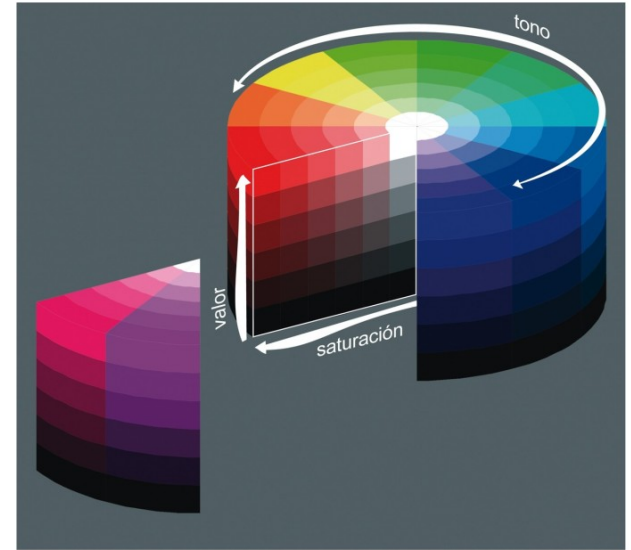
- HSV é a abreviatura para o sistema de cores formadas pelas componentes *hue* (matiz), *saturation* (saturação) e *value* (valor);
- O HSV também é conhecido como HSB (*hue*, *saturation* e *brightness* — matiz, saturação e brilho, respectivamente);
- Este sistema foi criado em 1974, por Alvy Ray Smith. É caracterizado por ser uma **transformação não-linear** do sistema de cores RGB.



Modelos de Representação

O Modelo HSV

- Esse sistema de cores define o espaço de cor conforme descrito abaixo, utilizando seus três parâmetros:
 - *Matiz* (tonalidade): verifica o tipo de cor, abrangendo todas as cores do espectro, desde o vermelho até o violeta, mais o magenta. Atinge valores de 0 a 360, mas para algumas aplicações, esse valor é normalizado de 0 a 100%;
 - *Saturação*: também chamado de "pureza". Quanto menor esse valor, mais com tom de cinza aparecerá a imagem. Quanto maior o valor, mais "pura" é a imagem. Atinge valores de 0 a 100%;
 - *Valor* (brilho): define o brilho da cor. Atinge valores de 0 a 100%.



Mapas de cores

Tabelas de alocação de cores

- Em sistemas gráficos, a tabela de alocação de cores (ou apenas tabela de cores ou *colormap*) é uma estrutura usada para mapear valores numéricos (índices) a cores reais (representadas em RGB ou outras codificações);
- Permite otimizar o uso de memória e simplificar o gerenciamento de cores, especialmente em dispositivos com paleta limitada ou em imagens indexadas;
- A tabela de cores funciona como um vetor de entradas, onde cada posição contém um valor RGB (ou outro modelo de cor). Ao invés de armazenar diretamente os valores de cor em cada entrada do *buffer* do SRD que representa a imagem a ser exibida no dispositivo gráfico real, armazena-se apenas o índice da cor na tabela.

Mapas de cores

Tabelas de alocação de cores

O uso de um *colormap* possui algumas vantagens, como:

- Redução do uso de memória: ideal para dispositivos com recursos limitados;
- Facilidade para alteração de cores em massa: basta atualizar a tabela;
- Utilizada em formatos de imagens com 8 bits por pixel.

Exemplo:

Índice	Cor (RGB)
0	(0, 0, 0) – Preto
1	(255, 0, 0) – Vermelho
2	(0, 255, 0) – Verde
3	(0, 0, 255) – Azul
4	(255, 255, 255) – Branco

Mapas de cores

Tabelas de alocação de cores

Display color map
image

1	5	3	2	2	4
5	2	4	2	5	1
5	5	5	5	3	3
2	1	2	4	1	3
4	4	4	1	1	3
2	4	2	1	3	3

Color map

	Red	Green	Blue
1	255	255	0
2	64	0	128
3	255	32	32
4	0	255	0
5	0	0	255